



Universidad Politécnica
de Madrid



**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Trabajo Fin de Grado

**APLICACION WEB para la GESTION y
RESERVA de SALAS de ESTUDIO de una
BIBLIOTECA**

Autor: Jorge Álvarez Montalvo

Tutor: Alejandro Rodríguez González

Madrid, mayo 2021



Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Ingeniería Informática

Título: APLICACION WEB para la GESTION y RESERVA de SALAS de
ESTUDIO de una BIBLIOTECA

Mayo 2021

Autor: Jorge Álvarez Montalvo

Tutor:

Alejandro Rodríguez González

Lenguajes y Sistemas Informáticos e Ingeniería del Software

ETSI Informáticos

Universidad Politécnica de Madrid



Resumen

La acción de reservar es una tarea muy común que lleva realizándose desde hace mucho tiempo. Se reservan artículos en tiendas, vehículos, mesas para un restaurante o habitaciones de hotel.

Sin embargo, los métodos de reserva que se venían utilizando desde hace algunas décadas, como lo son el acudir presencialmente al local, o llamar por teléfono, están quedando obsoletos. Obsoletos porque, en muchas ocasiones, causan problemas como no obtener respuesta desde el establecimiento o, en caso de necesitar acudir, no encontrar el momento para hacerlo.

Gracias a la evolución de la tecnología y la entrada en la era de la transformación digital, se han ido creando aplicaciones, como la desarrollada en este trabajo, que permiten a los usuarios realizar estas acciones sin la necesidad de depender del personal del establecimiento.

En este caso se trata de una aplicación que permita la reserva de las diferentes salas de estudio de una biblioteca de modo sencillo, gracias a su interfaz intuitiva y amigable para cualquier usuario.

Los usuarios podrán realizar, eliminar y modificar sus reservas siempre que estas no hayan caducado. Los administradores podrán gestionar las reservas realizadas, así como añadir, modificar y eliminar diferentes salas. Todo ello a través de la sencilla interfaz del sistema. El sistema cuenta, además, con varios mecanismos de seguridad que garantizan la integridad de los datos.

El resultado final es una aplicación totalmente funcional lista para su uso.

Palabras clave: Aplicación Web, Reserva, Diseño de API



Abstract

Booking is a very common task that has been carried out for a long time. People reserve items in shops, vehicles, tables in a restaurant or hotel rooms.

However, the booking methods that have been used for decades, such as going in person or calling by phone, are becoming obsolete. Obsolete because they often cause problems such as not getting a response from the establishment when calling, or not finding the right moment to get to the establishment when needed.

Thanks to the evolution of technology and the entry into the era of digital transformation, applications that allow users to carry out these actions without the need to depend on the establishment's staff have been created.

In this case, the application allows the user to book the different available study rooms in a library in a simple way thanks to its intuitive and user-friendly interface.

Users will be able to make, delete and modify their reservations only if they are not expired. Admins can manage the reservations made as well as add, modify, and delete the different rooms available to book. All of this can be done through the system's simple interface. To ensure the data integrity, the system also has several built-in security mechanisms.

The result is a fully functional application ready for use.

Keywords: Web Application, Booking, API Design



Tabla de contenidos

1	Introducción.....	1
1.1	Objetivos del proyecto	1
1.2	Planificación.....	2
2	Especificación de requisitos.....	4
3	Diseño	7
3.1	Modelo de datos y base de datos.....	7
3.2	Interfaz de usuario	11
3.3	Flujo de pantallas.....	12
3.4	API REST.....	13
4	Desarrollo	20
4.1	Proceso.....	20
4.2	Tecnologías	20
4.2.1	Lenguajes de programación.....	21
4.2.2	Back-end	21
4.2.3	Front-end.....	22
4.2.4	Auxiliares.....	23
4.3	Estructura del sistema	23
4.3.1	Back-end	24
4.3.2	Front-end.....	27
4.4	Seguridad.....	31
4.4.1	JWT	31
4.4.2	Base de datos: MySQL	33
4.4.3	Funcionalidad.....	34
5	Pruebas.....	35
6	Resultados y conclusiones	38
7	Análisis de impacto.....	40
8	Bibliografía	41
9	Anexos.....	42
9.1	Anexo 1 – Diseños de interfaz.....	42
9.2	Anexo 2 – Interfaz web.....	47



Índice de tablas

Tabla 1. Requisitos del sistema.....	4
Tabla 2. API. Registro de usuario.	13
Tabla 3. API. Login de usuario.	14
Tabla 4. API. GET de todos los usuarios.....	14
Tabla 5. API. GET de un usuario.....	14
Tabla 6. API. Editar información de usuario.....	14
Tabla 7. API. Editar contraseña de usuario.	15
Tabla 8. API. Eliminar usuario.....	15
Tabla 9. API. GET de todas las salas.	15
Tabla 10. API. GET de una sala.....	16
Tabla 11. API. Crear una sala.	16
Tabla 12. API. Editar datos de una sala.	16
Tabla 13. API. Eliminar una sala.....	16
Tabla 14. API. Habilitar una sala para su reserva.	17
Tabla 15. API. Deshabilitar una sala para su reserva.	17
Tabla 16. API. GET de todas las reservas.	17
Tabla 17. API. GET de una reserva.....	18
Tabla 18. API. GET de todas las reservas de un usuario.....	18
Tabla 19. API. GET de todas las reservas de una sala.	18
Tabla 20. API. Crear una reserva.	18
Tabla 21. API. Editar una reserva.	19
Tabla 22. API. Eliminar una reserva.....	19

Índice de ilustraciones

Ilustración 1. Diagrama de Gantt con la planificación inicial.....	2
Ilustración 2. Diagrama de Gantt con la planificación final.	3
Ilustración 3. Atributos de la entidad Usuario.....	7
Ilustración 4. Atributos de la entidad ‘Sala’.....	8
Ilustración 5. Atributos de la entidad ‘Reserva’.	9
Ilustración 6. Atributos de la entidad ‘Tipo de usuario’.	9
Ilustración 7. Atributos de la entidad 'Estado de una sala'.....	10
Ilustración 8. Relación entre entidades de la base de datos.....	10
Ilustración 9. Diseño de pantalla home.	11
Ilustración 10. Flujo de pantallas del sistema.	12
Ilustración 11. Modelo de funcionamiento del sistema.	24
Ilustración 12. Estructura del código del servidor.	24
Ilustración 13. Carpeta componentes del servidor.....	25
Ilustración 14. Contenido de la carpeta de un componente.	25
Ilustración 15. Fichero routes del componente reserva.....	25
Ilustración 16. Función del handler del componente reserva.....	26
Ilustración 17. Función del controller de reserva.....	26
Ilustración 18. Carpeta de configuración del servidor.....	27
Ilustración 19. Carpeta de utilidades del servidor.	27
Ilustración 20. Estructura del código del cliente.	27
Ilustración 21. Contenido del módulo público.	28
Ilustración 22. Ficheros de un componente.....	29
Ilustración 23. Método del fichero servicio de salas.	29
Ilustración 24. Rutas del fichero de routing.	30
Ilustración 25. Rutas con guards.	30
Ilustración 26. Estructura de un token JWT. [21]	31
Ilustración 27. Cabecera de JWT. [21].....	31
Ilustración 28. Carga de JWT. [21]	32
Ilustración 29. Firma de JWT. [21].....	32
Ilustración 30. Token JWT. [21]	32
Ilustración 31. Formato de cabecera ‘Authorization’ para token JWT. [21]	32
Ilustración 32. Proceso de obtención del token JWT.....	33



Ilustración 33. Cuerpo prueba petición inicio de sesión.	35
Ilustración 34. Respuesta prueba inicio de sesión.....	35
Ilustración 35. Autorización con token para las peticiones de las pruebas. ...	36
Ilustración 36. Formulario de prueba de inicio de sesión.	36
Ilustración 37. Cuerpo y URI enviados al servidor.....	36
Ilustración 38. Pantalla de login.....	38
Ilustración 39. Diseño de pantalla de inicio de sesión.	42
Ilustración 40. Diseño de pantalla de registro de usuario en el sistema.	42
Ilustración 41. Diseño de pantalla dashboard.	43
Ilustración 42. Diseño de pantalla de datos de usuario.	43
Ilustración 43. Diseño de pantalla de cambio de contraseña.	44
Ilustración 44. Diseño de pantalla de listado de salas	44
Ilustración 45. Diseño de pantalla de listado de reservas	45
Ilustración 46. Diseño de pantalla de reserva.....	45
Ilustración 47. Diseño de pantalla de error 404 - elemento no encontrado	46
Ilustración 48. Pantalla home.	47
Ilustración 49. Pantalla de registro.	47
Ilustración 50. Pantalla dashboard.	47
Ilustración 51. Pantalla de reservas con vista de administrador.	48
Ilustración 52. Pantalla de usuarios vista de administrador.....	48
Ilustración 53. Pantalla de salas vista de administrador.	48
Ilustración 54. Pantalla de creación de sala.	49
Ilustración 55. Pantalla de datos de usuario.	49
Ilustración 56. Pantalla de cambio de contraseña.	49
Ilustración 57. Pantalla de reserva.....	50
Ilustración 58. Pantalla de reservas de usuario.....	50
Ilustración 59. Pantalla de salas de usuario.....	50
Ilustración 60. Pantalla de error 404 Not Found.	51



1 Introducción

En un mundo en el que la tecnología avanza a pasos agigantados día tras día, es crucial que las empresas evolucionen para poder competir. La adaptación a esta nueva era tecnológica se conoce como la transformación digital y, a pesar de haber comenzado hace ya algunos años, aún quedan muchas empresas que no han iniciado dicha transición perdiendo así algunas de las numerosas ventajas que esta ofrece, como la oportunidad de incrementar su eficiencia.

En el pasado, al ir a realizar una reserva en un local, el interesado debía ponerse en contacto con el local y realizar la reserva, bien mediante teléfono o acudiendo presencialmente. Esta tarea era muchas veces tediosa por el tiempo de espera al teléfono, porque no había respuesta desde el propio establecimiento, o porque no se encontraba el momento de acudir a realizar la reserva.

Con el objetivo de facilitar este tipo de acciones y, siguiendo la corriente de la transformación digital, se empezaron a crear sistemas que permitiesen a los usuarios reservar sin la necesidad de contactar con el personal de un establecimiento, como es el caso de este proyecto.

En este documento se recoge la totalidad del proceso de creación de uno de estos sistemas, desde la obtención de requisitos y diseño, hasta el desarrollo y su implementación. En concreto, un sistema de reserva para las diferentes salas de estudio de las que disponga una biblioteca. No obstante, el sistema puede ser adaptado para ser utilizado para otro tipo de ámbitos en los que sea necesario realizar reservas, como pueden ser los restaurantes o los hoteles.

El sistema desarrollado permitirá a los usuarios realizar reservas sobre el conjunto de salas que el personal de la biblioteca ponga a su disposición, así como su gestión. El personal de la biblioteca podrá gestionar y administrar las diferentes salas que ofrezca a los usuarios, realizando acciones como bloquear las reservas para una sala determinada. Podrá también consultar las diferentes reservas realizadas por los usuarios y modificarlas en caso de que sea necesario.

1.1 Objetivos del proyecto

Para facilitar el seguimiento del proyecto, se han establecido unos objetivos que pueden ser usados como hitos. Los objetivos del proyecto son los siguientes:

- Analizar y capturar los requisitos para el sistema
- Diseñar el back-end del sistema
- Diseñar el front-end del sistema
- Desarrollar el sistema
- Testear el correcto funcionamiento del sistema
- Documentar el proceso en la memoria

1.2 Planificación

Para el control de las tareas a realizar durante las diferentes etapas de desarrollo del proyecto se estableció una planificación inicial. La planificación se ha representado mediante un diagrama de Gantt. La Ilustración 1 muestra la planificación inicial del proyecto.



Ilustración 1. Diagrama de Gantt con la planificación inicial.

Sin embargo, tras encontrar ciertos inconvenientes durante el proceso, no ha sido posible seguir la planificación inicial, teniendo que adaptarse a la nueva situación. Los principales inconvenientes encontrados son los siguientes:

- Pruebas y solución de errores del backend

Con el objetivo de asegurar la robustez y el buen funcionamiento del sistema, se realizaron numerosas pruebas en las que se detectaron comportamientos no esperados. Por este motivo, y por el tiempo que llevó solucionar los fallos detectados, esta tarea se alargó más de lo esperado afectando al resto de la planificación del proyecto.

- Desarrollo del frontend

A pesar del retraso causado por el desarrollo del backend, durante esta etapa también surgieron algunos problemas de funcionamiento. En especial, la implementación de la funcionalidad de reserva, concretamente el formulario que el usuario tiene que rellenar para realizarla. A causa de esto, la planificación se vio de nuevo retrasada.

- Pruebas generales y solución de errores del sistema

Al igual que con el backend y el frontend, se realizaron numerosas pruebas para asegurar el correcto funcionamiento del sistema de forma global. En ellas no se encontraron comportamientos inesperados, pero el tiempo que fue necesario para realizarlas fue más del esperado.

La Ilustración 2 muestra la planificación final del proyecto incluyendo las nuevas fechas. Las partes de la planificación inicial que se vieron alteradas quedan marcadas con un cuadro rojo en la imagen.

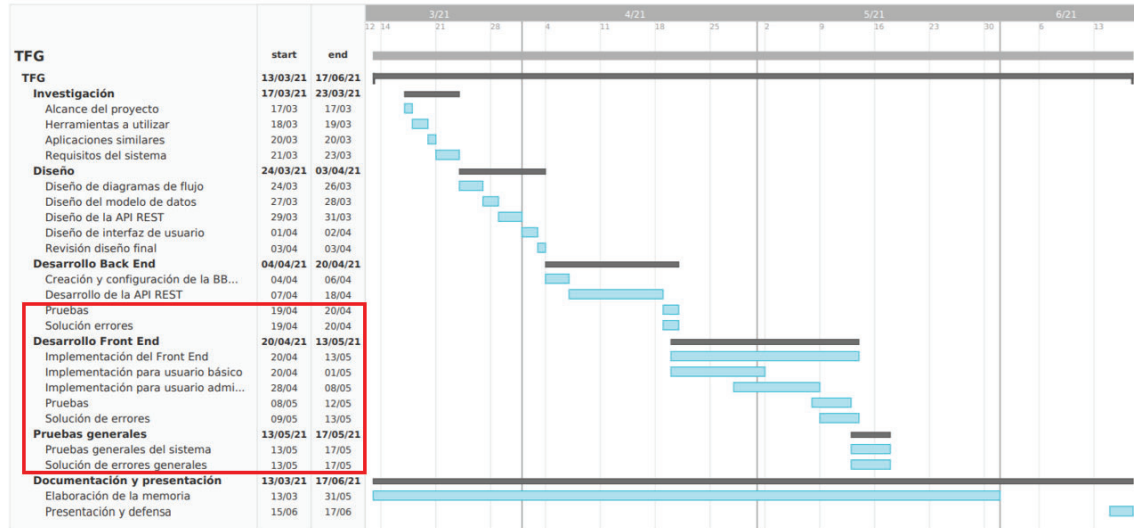


Ilustración 2. Diagrama de Gantt con la planificación final.

2 Especificación de requisitos

A lo largo de esta sección se detallan los diferentes requisitos del sistema. Estos requisitos han sido tomados como base para el desarrollo de las distintas funcionalidades de las que dispone la aplicación web.

Con el objetivo de facilitar el desarrollo de la aplicación web, los requisitos del sistema se han agrupado en diferentes categorías en función de a qué funcionalidad hacen referencia, permitiendo así dividir el sistema en varios módulos.

En la Tabla 1 se recogen, a modo de resumen, todos los requisitos del sistema agrupados en los mencionados módulos.

Funcionalidad	Identificador de requisito	Requisito
SESION/ACCESO	R 01	Registro de usuario
	R 02	Inicio de sesión de usuario
	R 03	Fin de sesión de usuario
USUARIO	R 04	Consulta de usuarios
	R 05	Consulta de datos de usuario
	R 06	Editar datos de usuario
	R 07	Eliminar usuario
SALAS	R 08	Consulta de salas
	R 09	Añadir sala
	R 10	Modificar sala
	R 11	Eliminar sala
	R 12	Disponibilidad de sala
RESERVAS	R 13	Realizar reserva
	R 14	Consulta de reservas
	R 15	Editar reserva
	R 16	Eliminar reserva

Tabla 1. Requisitos del sistema

Para asegurar la comprensión de los requisitos, se detallará cada uno de ellos. Previamente, se definen algunos términos relevantes:

- **Usuario:** el usuario es la persona o individuo que va a acceder al sistema, la que tiene la capacidad de, tras registrarse, realizar una reserva. El sistema tiene dos tipos de usuarios: el usuario básico, que puede hacer uso de funcionalidades básicas, y el usuario administrador, con acceso a todas las funcionalidades del sistema, como la gestión de las salas.



- **Sala:** se entiende como sala, en este caso, una habitación de la biblioteca disponible para su reserva.

A continuación, se detalla cada uno de los requisitos:

- R 01 - Registro de usuario

Un usuario puede registrarse en el sistema con el fin de utilizar la funcionalidad de reserva de salas. Para ello, deberá rellenar un formulario incluyendo un usuario y contraseña para iniciar sesión y su correo. De manera opcional podrá especificar su nombre.

- R 02 - Inicio de sesión de usuario

Un usuario podrá iniciar sesión en el sistema introduciendo un usuario y contraseña correctos.

- R 03 - Fin de sesión de usuario

Un usuario podrá cerrar sesión siempre y cuando tenga una sesión activa válida.

- R 04 - Consulta de usuarios

Un usuario de tipo administrador podrá realizar una consulta sobre el sistema obteniendo una lista de todos los usuarios, tanto básicos como administradores, registrados en el sistema.

- R 05 - Consulta de datos de usuario

Un usuario básico podrá consultar los datos de su perfil y únicamente de su perfil. El usuario de tipo administrador podrá obtener los datos de cualquier tipo de usuario.

- R 06 - Editar datos de un usuario

Un usuario básico podrá modificar algunos de los datos de su perfil, como el nombre o la contraseña. En cualquier caso, no se podrá modificar el usuario utilizado para iniciar sesión ni el correo asociado a la cuenta. Un usuario de tipo administrador podrá modificar datos de cualquier usuario, teniendo las mismas restricciones que estos respecto al usuario y correo.

- R 07 - Eliminar usuario

Un usuario administrador podrá eliminar cualquier usuario del sistema siempre y cuando este no tenga una sesión activa.

- R 08 - Consulta de salas

Los usuarios del sistema podrán realizar una consulta obteniendo un listado de todas las salas disponibles en el sistema.

- R 09 - Añadir sala

Un usuario administrador podrá añadir una nueva sala al sistema a través de un formulario, en el que tendrá que rellenar los siguientes campos: el nombre de la sala, el número de plazas que esta tiene, la planta en la que se encuentra y una pequeña descripción si fuese necesario.



- R 10 - Modificar sala

Un usuario administrador podrá modificar los datos de cualquier sala existente en el sistema.

- R 11 - Eliminar sala

Un usuario administrador podrá eliminar cualquier sala existente en el sistema.

- R 12 - Disponibilidad de sala

Un usuario administrador podrá habilitar y deshabilitar una sala con el objetivo de que esta no aparezca disponible para ser reservada por el resto de los usuarios del sistema.

- R 13 - Realizar reserva

Los usuarios del sistema podrán realizar una reserva sobre aquellas salas disponibles en el momento de realizar la misma. Para ello, deberán rellenar un formulario con los siguientes campos: el día de reserva de la sala, la hora de inicio de la reserva y el número de horas que quieren reservar.

- R 14 - Consulta de reservas

Un usuario básico podrá realizar una consulta al sistema obteniendo las reservas que haya realizado hasta el momento. Un usuario administrador podrá además obtener todas las reservas realizadas en el sistema hasta la fecha de la consulta.

- R 15 - Editar reserva

Un usuario básico podrá editar una de sus reservas. Un usuario administrador podrá editar cualquier reserva. Las reservas podrán ser editadas siempre y cuando sigan siendo válidas. Esto es, que la fecha de inicio de la reserva sea posterior a la fecha en que se consulta la reserva.

- R 16 - Eliminar reserva

Un usuario básico podrá eliminar una de sus reservas. Un usuario administrador podrá eliminar cualquier reserva del sistema.

3 Diseño

A lo largo de este apartado se detallan los diferentes elementos y decisiones de diseño que se han tomado para el desarrollo del sistema. Este apartado se encuentra dividido en varios subapartados: el Modelo de Datos y Base de Datos, la Interfaz de Usuario, el Flujo de Pantallas y la API.

3.1 Modelo de datos y base de datos

El concepto principal del sistema es permitir a los usuarios realizar reservas sobre un conjunto de salas de una biblioteca. Para alcanzar este objetivo, se ha determinado que el sistema tenga las siguientes entidades:

- Usuario
- Sala
- Reserva
- Tipo de usuario
- Estado de la sala

A continuación, se detalla cada una de ellas.

❖ USUARIO

El usuario representa al individuo que interactúa con el sistema. El conjunto de atributos que lo forman se puede observar en la Ilustración 3.

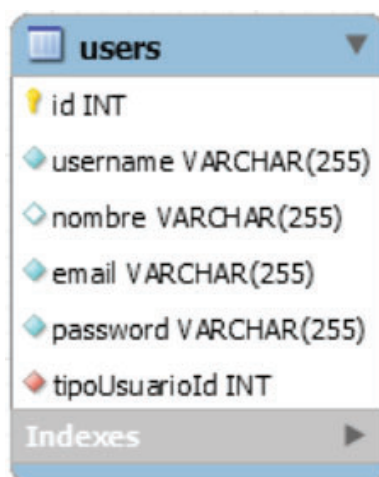


Ilustración 3. Atributos de la entidad Usuario.

- **id**: Es el identificador único de cada usuario en la base de datos. Es la clave primaria (PK).
- **username**: Representa el nombre de usuario del individuo, el nombre con el que hace inicio de sesión y con el que la aplicación le identifica. Es único.

- **nombre:** Representa el nombre del individuo. Es un valor opcional en el proceso de registro.
- **email:** Correo electrónico del usuario. Es único.
- **password:** Contraseña del usuario para el inicio de sesión en el sistema. La contraseña que el usuario introduce al crear la cuenta es almacenada en la base de datos cifrada.
- **tipoUsuarioId:** Representa el tipo de usuario en el sistema haciendo referencia a otra entidad en la que se definen los diferentes tipos. Es una clave foránea (FK).

❖ SALA

La sala representa a cada espacio disponible para reservar en la biblioteca. Los atributos que la componen se pueden observar en la Ilustración 4.

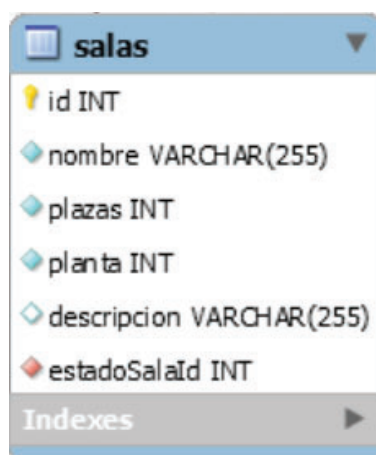


Ilustración 4. Atributos de la entidad 'Sala'.

- **id:** Identificador único de cada sala en la base de datos. Es la clave primaria (PK).
- **nombre:** Representa el nombre que tiene una sala (ej. Sala 1, Sala Verde...).
- **plazas:** Representa la ocupación máxima de una sala.
- **planta:** Representa la planta del edificio en el que se encuentra la sala.
- **descripción:** Pequeña descripción que puede dar el personal a una sala para dar algo más de información. Es opcional a la hora de la creación de nuevas salas en el sistema.
- **estadoSalaId:** Representa el estado en el que se encuentra una sala haciendo referencia a otra tabla en la que se definen los diferentes estados. Es una clave foránea (FK).

❖ RESERVA

La reserva es la entidad encargada de recoger la información sobre las distintas reservas que los usuarios realizan sobre las salas. Los atributos que la forman se pueden observar en la Ilustración 5.



Ilustración 5. Atributos de la entidad 'Reserva'.

- **id:** Identificador único de una reserva en la base de datos. Es la clave primaria (PK).
- **nombre:** Nombre de la reserva.
- **día:** Fecha de la reserva en formato YYYY-MM-DD.
- **hora_inicio:** Hora de inicio de la reserva en formato HH:MM.
- **duracion:** Duración de la reserva. Valor por defecto de 1h siendo el máximo de 2 horas.
- **descripcion:** Descripción del usuario sobre la reserva. Es opcional a la hora de crear la reserva.
- **salaId:** Identificador que representa la sala que se está reservando. Es una clave foránea (FK).
- **userId:** Identificador que representa el usuario que está reservando. Es una clave foránea (FK).

❖ TIPO DE USUARIO

El tipo de usuario es la entidad que se utiliza para determinar el rango que tiene el usuario en el sistema y así limitar las funcionalidades que este tiene disponibles. Se ha escogido implementarlo como una entidad para, en un futuro, poder implementar más tipos de usuarios al sistema. Los atributos pueden observarse en la Ilustración 6.

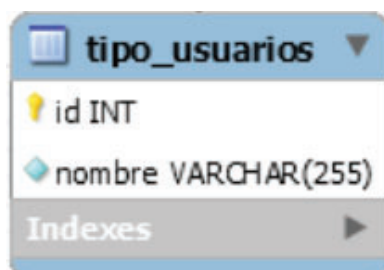


Ilustración 6. Atributos de la entidad 'Tipo de usuario'.

- **id**: Identificador único del tipo de usuario en la base de datos. Es la clave primaria (PK).
- **nombre**: Nombre del tipo de usuario.

❖ ESTADO DE LA SALA

El estado de la sala representa el estado actual en el que se encuentra una sala. Al igual que el tipo de usuario, se ha decidido implementar como entidad para permitir en un futuro añadir varios tipos de estado si fueran necesarios. Los atributos que forman esta entidad se representan en la Ilustración 7.

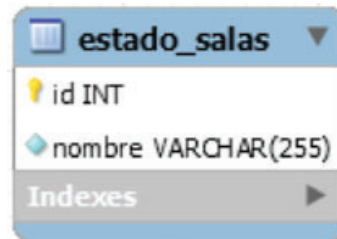


Ilustración 7. Atributos de la entidad 'Estado de una sala'.

Una vez definidas las entidades que forman la base de datos, en la Ilustración 8 se muestran las relaciones existentes entre ellas.

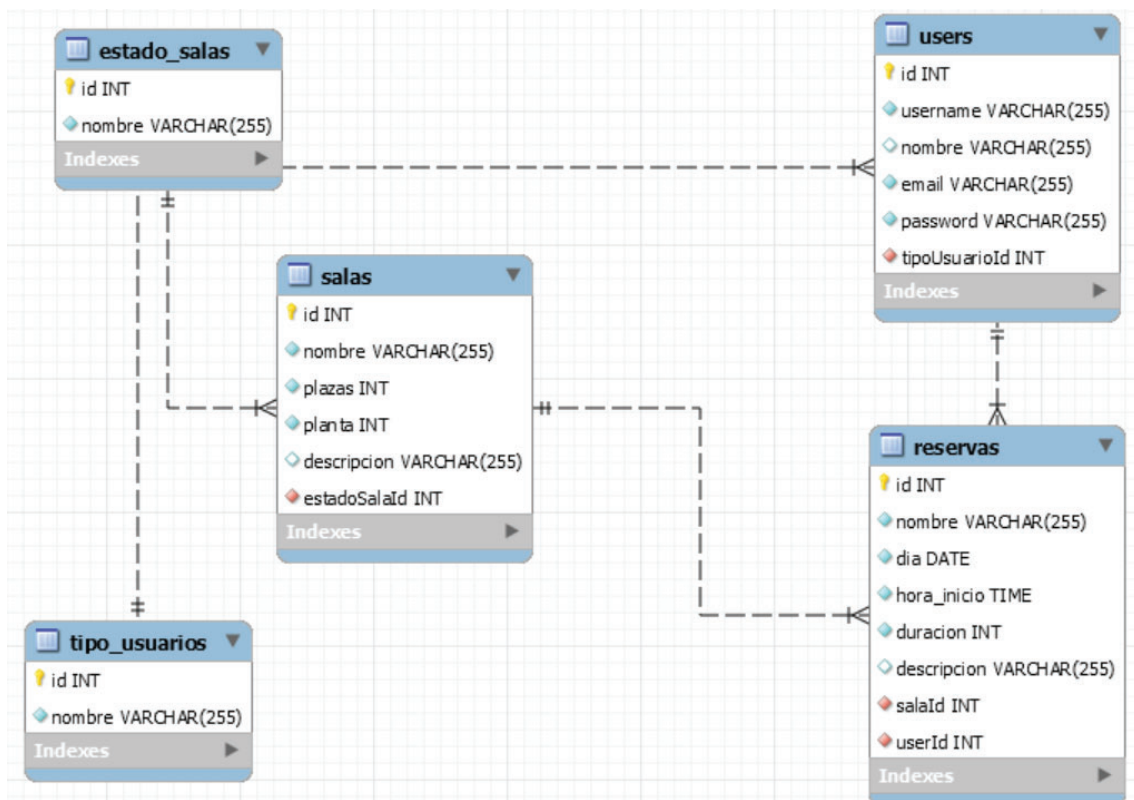


Ilustración 8. Relación entre entidades de la base de datos.

3.2 Interfaz de usuario

A lo largo de las diferentes pantallas, se ha intentado mantener un estilo en el que el usuario disponga de toda la información y acciones posibles visibles en pantalla, evitando así que tenga que desplazarse utilizando la barra lateral del navegador.

La pantalla de inicio 'home' es la siguiente:

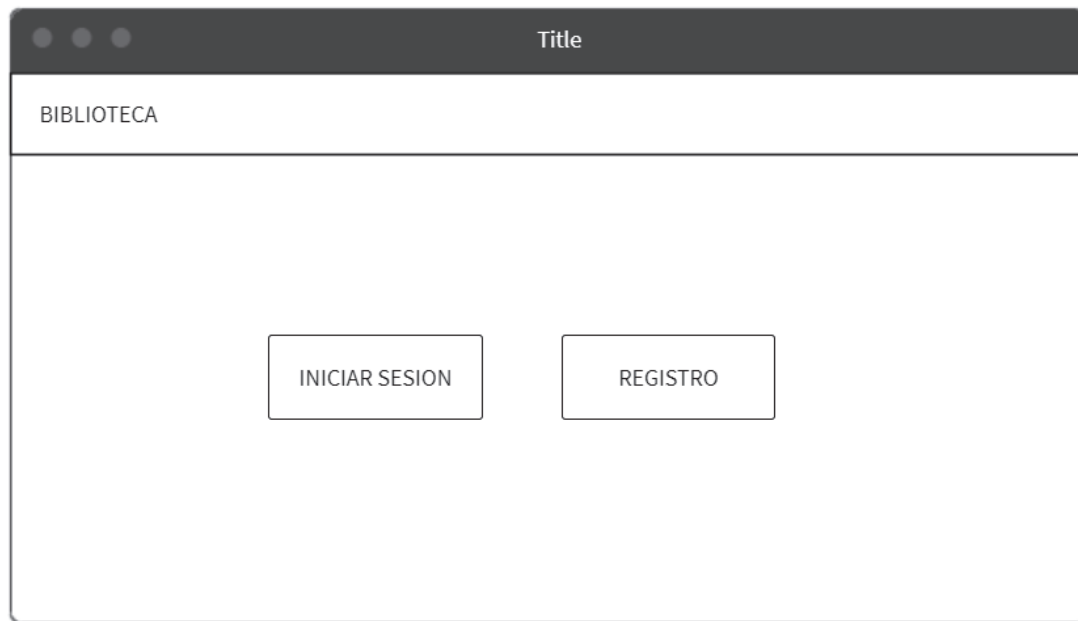


Ilustración 9. Diseño de pantalla home.

Los diseños de la interfaz que los usuarios van a visualizar al acceder al sistema de reserva web pueden encontrarse en el 'Anexo 1 – Diseños de interfaz'.

3.3 Flujo de pantallas

La Ilustración 10 muestra el flujo de navegación entre las diferentes pantallas de las que dispone el sistema de reserva web.

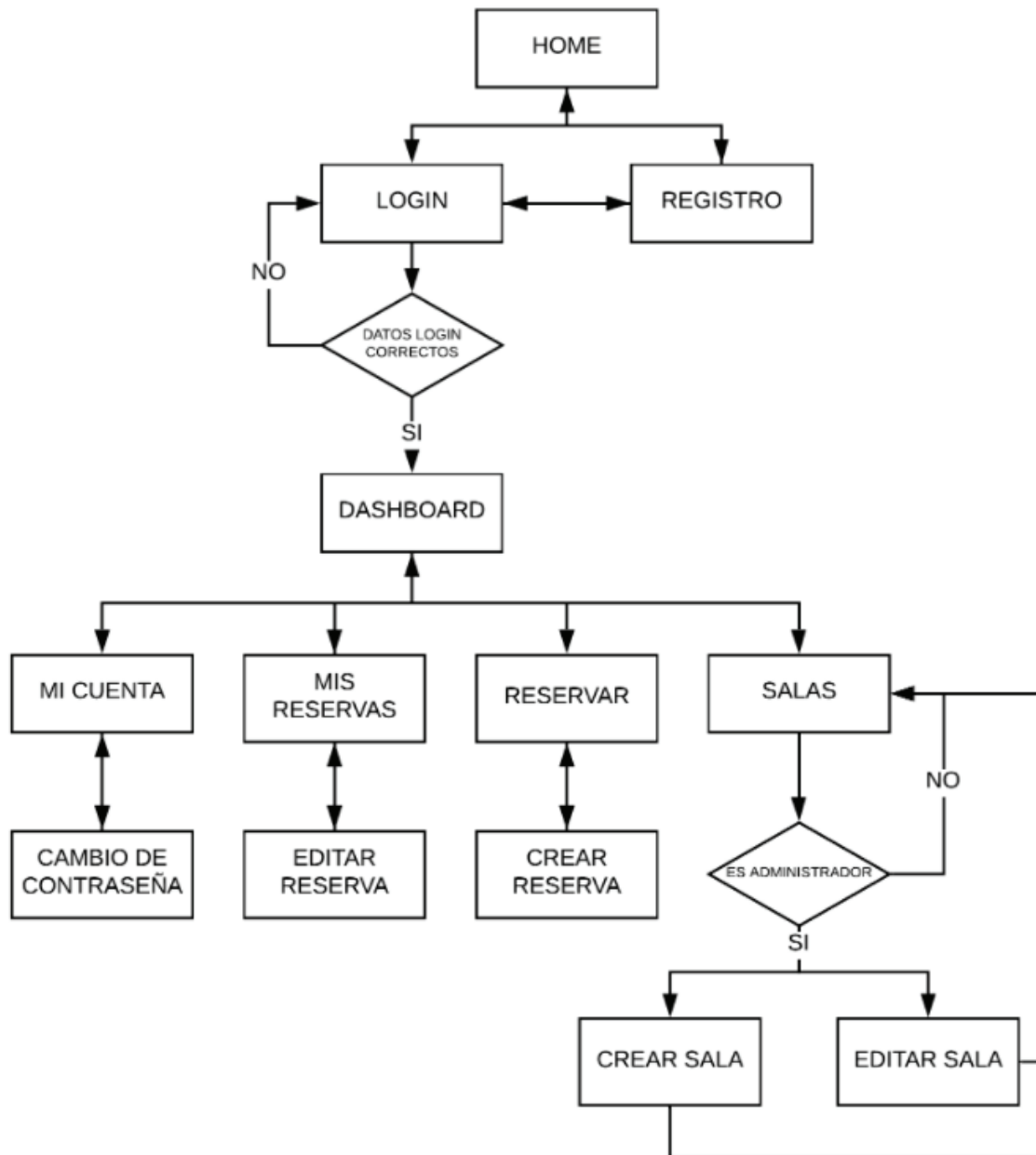


Ilustración 10. Flujo de pantallas del sistema.

3.4 API REST

La API contiene toda la lógica necesaria para el correcto funcionamiento del sistema, esto es, operar adecuadamente según lo esperado. La API es también la encargada del manejo de los datos.

La API seguirá una arquitectura REST, un tipo de arquitectura software apoyada en el protocolo HTTP en el que:

- Todo es un recurso con un identificador único, su URI
- Cada recurso puede representarse de múltiples formas: json, xml o yaml
- La comunicación no tiene estado, ya que en cada petición se incluye toda la información necesaria
- Se opera sobre los recursos haciendo uso de los verbos de HTTP como lo son GET, POST, PUT o DELETE

Para cada punto de acceso, se detallarán los siguientes aspectos:

- URI: corresponde al identificador único del recurso que se quiere utilizar.
- Método: corresponde al método HTTP de la operación, siendo alguno de los siguientes: GET, POST, PUT o DELETE.
- Descripción: descripción del recurso y posibles errores¹.
- Código de respuesta: código de respuesta HTTP que devuelve la API en función del resultado de la petición realizada.

Al igual que con los requisitos del sistema, la API puede ser dividida en grupos en función de la funcionalidad de estos. En este caso se dividirá en 3 grupos: USUARIO, SALAS y RESERVAS.

❖ USUARIOS

En este subapartado se detallan todos los puntos de acceso referentes a los usuarios del sistema.

- Registro de un usuario

URI	/register	
Método	POST	
Descripción	Crea un usuario en el sistema tras rellenar el formulario de registro (username, password, email, nombre). En caso de ya existir un usuario con el mismo username o email el sistema no permite el registro y envía un error.	
Código de respuesta	200	Registro de usuario correcto
	400	Nombre de usuario o email ya en uso.

Tabla 2. API. Registro de usuario.

¹ Ciertos puntos de acceso devuelven el error 403-Forbidden cuando el usuario no dispone de los permisos suficientes para realizar una acción. En la tabla no se especifican porque dependen de una comprobación previa y no del punto de acceso.

- Inicio de sesión de un usuario

URI	/login	
Método	POST	
Descripción	Permite a un usuario iniciar sesión en el sistema. En caso de no existir el usuario o mandar un par username-password incorrecto el sistema envía un error.	
Código de respuesta	200	Inicio de sesión correcto
	403	Usuario o contraseña incorrectos
	404	Usuario no encontrado
	500	Error generando el token de acceso

Tabla 3. API. Login de usuario.

- Obtener la información de todos los usuarios

URI	/users	
Método	GET	
Descripción	Devuelve una lista con la información (username, email, nombre, tipoUsuario) de cada usuario del sistema.	
Código de respuesta	200	Solicitud correcta

Tabla 4. API. GET de todos los usuarios.

- Obtener la información de un usuario

URI	/users/{username}	
Método	GET	
Descripción	Devuelve la información (username, email, nombre, tipoUsuario) del usuario {username}. En caso de no encontrar el usuario el sistema devuelve un error.	
Código de respuesta	200	Solicitud correcta
	404	Usuario no encontrado

Tabla 5. API. GET de un usuario.

- Editar la información de un usuario

URI	/users/{username}	
Método	PUT	
Descripción	Edita la información (nombre) del usuario {username}. En caso de no existir o encontrar un problema actualizando los datos el sistema devuelve un error.	
Código de respuesta	200	Datos editados correctamente
	404	Usuario no encontrado
	500	Error al editar datos

Tabla 6. API. Editar información de usuario.

- Modificar la contraseña de un usuario

URI	/users/{username}/changepassword	
Método	PUT	
Descripción	Edita la contraseña del usuario {username}. Para editar la contraseña se debe proporcionar la antigua y una nueva. En caso de que la contraseña antigua no coincida, que el usuario no exista o que haya error actualizando los datos el sistema envía un error.	
Código de respuesta	200	Contraseña modificada correctamente
	403	Contraseña antigua no coincide
	404	Usuario no encontrado
	500	Error actualizando contraseña

Tabla 7. API. Editar contraseña de usuario.

- Eliminar un usuario

URI	/users/{username}	
Método	DELETE	
Descripción	Elimina al usuario {username} del sistema. En caso de que el usuario tenga sesión iniciada el sistema envía un error.	
Código de respuesta	200	Usuario eliminado correctamente
	403	Error eliminando usuario. Usuario con sesión iniciada

Tabla 8. API. Eliminar usuario.

❖ SALAS

En este subapartado se detallan todos los puntos de acceso referentes a las salas del sistema.

- Obtener la información de todas las salas

URI	/salas	
Método	GET	
Descripción	Devuelve una lista con todas las salas del sistema	
Código de respuesta	200	Solicitud correcta

Tabla 9. API. GET de todas las salas.



- Obtener la información de una sala

URI	/salas/{sala_id}	
Método	GET	
Descripción	Obtiene la información de una sala con el id {sala_id}. En caso de no existir el sistema devuelve un error.	
Código de respuesta	200	Solicitud correcta
	404	Sala no encontrada

Tabla 10. API. GET de una sala.

- Crear una sala

URI	/salas	
Método	POST	
Descripción	Crea una nueva sala en el sistema tras rellenar el formulario correspondiente (nombre, plazas, piso, descripción). En caso de producirse un error en la creación el sistema envía un error.	
Código de respuesta	200	Sala creada correctamente
	500	Error creando la sala

Tabla 11. API. Crear una sala.

- Editar los datos de una sala

URI	/salas/{sala_id}	
Método	PUT	
Descripción	Edita los datos de una sala (nombre, plazas, planta, descripción). En caso de que la sala no exista o se produzca un error actualizando los datos el sistema devuelve un error.	
Código de respuesta	200	Sala editada correctamente
	404	Sala no encontrada
	500	Error actualizando datos de la sala

Tabla 12. API. Editar datos de una sala.

- Eliminar una sala

URI	/salas/{sala_id}	
Método	DELETE	
Descripción	Elimina la sala con id {sala_id} del sistema.	
Código de respuesta	200	Sala eliminada correctamente

Tabla 13. API. Eliminar una sala.

- Habilitar una sala

URI	/salas/{sala_id}/habilitar	
Método	PUT	
Descripción	Edita el estado de una sala habilitándola para ser reservada. En caso de no encontrar la sala o no poder actualizar el estado el sistema devuelve un error.	
Código de respuesta	200	Sala habilitada correctamente
	404	Sala no encontrada
	500	Error actualizando estado de sala

Tabla 14. API. Habilitar una sala para su reserva.

- Deshabilitar una sala

URI	/salas/{sala_id}/deshabilitar	
Método	PUT	
Descripción	Edita el estado de una sala deshabilitándola para evitar ser reservada. En caso de no encontrar la sala o no poder actualizar el estado el sistema devuelve un error.	
Código de respuesta	200	Sala deshabilitada correctamente
	404	Sala no encontrada
	500	Error actualizando estado de sala

Tabla 15. API. Deshabilitar una sala para su reserva.

❖ RESERVAS

En este subapartado se detallan todos los puntos de acceso referentes a las reservas del sistema.

- Obtener la información de todas las reservas

URI	/reservas	
Método	GET	
Descripción	Devuelve la información de todas las reservas del sistema.	
Código de respuesta	200	Solicitud correcta

Tabla 16. API. GET de todas las reservas.

- Obtener la información de una reserva

URI	/reservas/{reserva_id}	
Método	GET	
Descripción	Obtiene la información de la reserva con el id {reserva_id}. En caso de no existir el sistema devolverá un error.	
Código de respuesta	200	Petición correcta
	404	Reserva no encontrada

Tabla 17. API. GET de una reserva

- Obtener las reservas de un usuario

URI	/reservas/users/{username}	
Método	GET	
Descripción	Obtiene todas las reservas que ha realizado el usuario {username}. El sistema devuelve un error en caso de no existir el usuario.	
Código de respuesta	200	Solicitud correcta
	404	El usuario no existe

Tabla 18. API. GET de todas las reservas de un usuario.

- Obtener las reservas de una sala

URI	/reservas/salas/{sala_id}	
Método	GET	
Descripción	Obtiene todas las reservas que se han realizado sobre la sala con id {sala_id}. En caso de no existir reservas en la sala, el sistema devuelve una lista vacía.	
Código de respuesta	200	Solicitud correcta

Tabla 19. API. GET de todas las reservas de una sala.

- Crear una reserva

URI	/reservas/nueva	
Método	POST	
Descripción	Crea una reserva tras rellenar un formulario (nombre, día, hora de inicio, duración, descripción). En caso de que el día y la hora se repitan para una sala determinada, la sala no exista o se produzca un error en la creación de la reserva, el sistema devuelve un error.	
Código de respuesta	200	Reserva creada correctamente
	404	Sala de reserva no encontrada
	409	Reserva ya existente
	500	Error creando la reserva

Tabla 20. API. Crear una reserva.



- Editar una reserva

URI	/reservas/{reserva_id}	
Método	PUT	
Descripción	Modifica los datos de una reserva con id {reserva_id}. En caso de que no encuentre la reserva, el usuario no sea el administrador o el dueño de la reserva, la fecha de la reserva sea anterior a la fecha del día de la edición o se produzca un error editando los datos, el sistema envía un error.	
Código de respuesta	200	Reserva modificada correctamente
	403	Usuario no autorizado
	404	Reserva no encontrada
	500	Error actualizando datos

Tabla 21. API. Editar una reserva.

- Eliminar una reserva

URI	/reservas/{reserva_id}	
Método	DELETE	
Descripción	Elimina la reserva con el id {reserva_id}	
Código de respuesta	200	Reserva eliminada correctamente

Tabla 22. API. Eliminar una reserva.

4 Desarrollo

A lo largo de este apartado se describe el proceso que se ha seguido durante el desarrollo de la aplicación web, las tecnologías utilizadas, así como algunos aspectos de la seguridad presentes en el sistema.

4.1 Proceso

La primera tarea a realizar ha sido la captura de los requisitos de la aplicación para así establecer los límites del sistema y las funcionalidades que este va a tener. Estos requisitos quedan definidos en el apartado ‘2. Especificación de requisitos’.

Una vez que los requisitos quedaron claramente definidos, el siguiente paso ha sido comenzar con el diseño del modelo de datos que va a utilizar la aplicación, así como la base de datos en la que se almacenará la información de forma persistente. Ambos pueden ser consultados en el apartado ‘3.1 Modelo de datos y base de datos’. En este punto también se ha realizado una primera aproximación al diseño de la interfaz del usuario y la navegación entre las diferentes pantallas.

Diseñados el modelo de datos y la base de datos, se comenzó con el desarrollo de la parte del servidor o back-end, que queda detallado en el apartado ‘4.3.1 Back-end’. El primer paso fue el diseño de la API, que contiene toda la lógica de negocio y es la encargada de hacer efectivas las funcionalidades definidas en los requisitos del sistema. El diseño ha consistido en crear los diferentes endpoints o puntos de acceso sobre los que se realizarán las peticiones a los recursos del sistema. Una vez diseñados los puntos de acceso para los recursos, se ha implementado la lógica de negocio y la conexión con la base de datos.

Finalizada la parte del back-end, se han realizado diferentes pruebas para asegurar el funcionamiento esperado haciendo uso de la herramienta Postman, detallada más adelante en el apartado ‘4.2.4 Auxiliares’.

Terminado el desarrollo del back-end, quedaba la tarea de diseñar y desarrollar la interfaz con las que va a interactuar el usuario, el cliente o front-end, de la que ya se tenía una primera aproximación. Completado el diseño, que puede encontrarse en el apartado ‘3.2 Interfaz de usuario’, se inició su desarrollo, que se encuentra detallado en el apartado ‘4.3.2 Front-end’.

Una vez terminado el desarrollo, se han realizado numerosas pruebas asegurando que el funcionamiento global del sistema, así como los datos enviados y recibidos entre ambas partes era el esperado.

4.2 Tecnologías

A lo largo de este apartado, se describen las diferentes tecnologías utilizadas para cada parte del sistema. El conjunto de tecnologías utilizadas para el desarrollo del sistema de reservas está en la vanguardia, ya que todas ellas son utilizadas por multitud de empresas en el desarrollo de proyectos y aplicaciones profesionales.



4.2.1 Lenguajes de programación

JavaScript

JavaScript [1] (JS) es un lenguaje de programación ligero, débilmente tipado y compilado en tiempo de ejecución. Habitualmente es utilizado en el lado del cliente como lenguaje de scripting en páginas web, pero también puede ser utilizado en entornos fuera del navegador actuando como servidor.

Typescript

Typescript [2] (TS) es un lenguaje de programación basado en JavaScript que nace de la necesidad de otorgar tipos a las diferentes variables y estructuras de datos con las que tratamos. Al estar fuertemente tipado, facilita la validación y el correcto funcionamiento del código, especialmente en proyectos de gran envergadura en los que existen multitud de estructuras de datos.

HTML

HTML [3] (HyperText Markup Language) es el lenguaje encargado de definir el significado y la estructura del contenido de un sitio web. Para ello, hace uso de diferentes etiquetas.

CSS

CSS [4] (Cascading Style Sheets) es un lenguaje de estilos encargado de dotar de estilo y describir la presentación de documentos HTML, desde el posicionamiento en la página hasta el tamaño o color. Permite establecer diferentes formas de presentación en función del dispositivo en el que se visualice una determinada página web.

4.2.2 Back-end

El back-end contiene prácticamente la totalidad de lógica de negocio de la aplicación. Es el encargado de acceder a los datos almacenados en la base de datos y de manejar las distintas peticiones que recibe desde el front-end, así como de enviar las respuestas a las peticiones recibidas. Para cumplir con estas funciones, se han escogido las siguientes tecnologías:

- NodeJS
- MySQL
- Redis

NodeJS

NodeJS [5] es un entorno de ejecución para el lenguaje de programación de JavaScript. Basado en el motor v8 de Google, está orientado a eventos asíncronos y fue diseñado especialmente para permitir el escalado de programas en la red como los servidores web.

A diferencia de otros entornos, NodeJS presenta una ejecución de un único hilo en el que no existe una llamada bloqueante para entrar en el bucle de ejecución. En su lugar, cada vez que NodeJS ejecuta un script de entrada, entra en el bucle



de ejecución del que solo saldrá una vez no queden más callbacks, que es el retorno a la llamada realizada. La forma en que opera permite además que los procesos nunca queden bloqueados.

NodeJS cuenta además con un gestor de paquetes, NPM [6], con multitud de librerías que facilitan el desarrollo de aplicaciones. Algunas de las utilizadas en el desarrollo de este sistema son:

- Express [7]: marco de aplicaciones web para el back-end que permite crear un servidor http de forma rápida y sencilla.
- Sequelize [8]: ORM (Object Relational Mapping o mapeo de objeto-relacional) que permite la gestión y administración de bases de datos de diferentes tipos como MySQL, Postgres o MariaDB.
- Jsonwebtoken: paquete que permite el uso sencillo de los JSON Web Token, detallados más adelante en el apartado '4.4.1 JWT'.

MySQL

Para el almacenamiento de los datos de la aplicación de forma persistente se ha optado por MySQL [10], un SGBDR o Sistema de Gestión de Bases de Datos Relacionales [11]. Puntualmente se ha hecho uso de MySQL Workbench [12], interfaz gráfica de MySQL que permite de forma visual diseñar y gestionar bases de datos.

Redis

Redis [13](REmote DIctionary Server), es un sistema de almacenamiento en memoria basado en el almacenamiento en tablas de hash. Almacena pares tipo clave-valor y es, en la actualidad, el sistema de almacenamiento de este tipo más popular [14]. Se ha escogido principalmente para la funcionalidad de sesión de usuario, por la capacidad que tiene de establecer un tiempo de vida a los pares clave-valor de información que almacena, permitiendo así de forma sencilla un tiempo de sesión.

4.2.3 Front-end

El front-end o parte cliente es la encargada de interactuar con el usuario y de enviar las diferentes solicitudes al servidor. Es también la encargada de mostrar por pantalla, a través de una interfaz, los diferentes datos que recibe del servidor. Para esta parte del sistema, las tecnologías escogidas han sido:

- Angular
- Angular Material

Angular

Angular [15] es un framework de código abierto desarrollado y mantenido por Google en el lenguaje de programación Typescript. Es utilizado para crear y mantener aplicaciones web de una sola página, emulando las aplicaciones de escritorio para otorgar al usuario una experiencia más fluida. Angular fue desarrollada con el objetivo de incrementar el número de aplicaciones de



navegador basadas en la arquitectura Modelo Vista Controlador [16] (MVC), en el que la lógica, los datos y su representación están separados.

Angular Material

Angular Material [17] es una librería de componentes de interfaz y estilos desarrollada por el equipo de Angular, pensada para integrarse de forma perfecta con el framework de Angular.

4.2.4 Auxiliares

Github

Con el objetivo de facilitar el seguimiento del desarrollo, así como de tener copias de seguridad del código, se ha utilizado Github [18], un servicio en la nube que permite llevar un registro y control sobre los cambios realizados en los diferentes ficheros del código, así como almacenarlo en un repositorio² en la nube. Github hace uso de Git [19] un software que permite el control de versiones creado por el ingeniero de software Linus Torvalds, también conocido por desarrollar el kernel del sistema operativo Linux.

Postman

Postman [20] es una aplicación que comenzó como extensión del navegador de Google Chrome que permite el envío de peticiones HTTP de tipo REST sin la necesidad de disponer de un cliente. Permite configurar una gran cantidad de parámetros para realizar test sobre puntos de entrada a una API.

VSCode

Para escribir la totalidad del código se ha usado VSCode [21] o Visual Studio Code, un editor de código desarrollado por la empresa Microsoft. Se ha escogido por las amplias opciones que dispone para desarrolladores como la integración de Git, el autocompletado y sugerencias de código, así como su amplio sistema de extensiones.

4.3 Estructura del sistema

El sistema está dividido en tres partes claramente diferenciadas: el cliente, el servidor y la base de datos. Si tomamos el servidor y la base de datos en conjunto, el sistema sigue un modelo de cliente-servidor.

² Espacio centralizado donde se almacena de forma organizada y se difunde información. En el caso de un repositorio informático, la información se corresponde con archivos informáticos digitales.

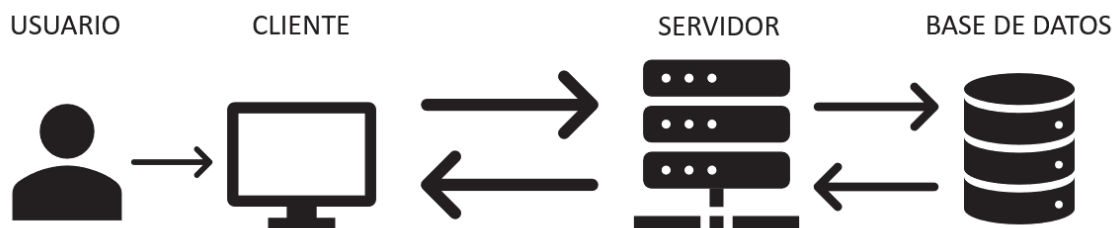


Ilustración 11. Modelo de funcionamiento del sistema.

En él, toda la lógica de negocio reside en la parte del servidor, que se encarga de atender las diferentes peticiones que los usuarios realizan y de enviar una respuesta. En caso de necesitar enviar datos, se encarga de traerlos de la base de datos.

El cliente, por su parte, es el encargado de la interacción con el usuario y de enviar diferentes peticiones al servidor. Es también el encargado de mostrar los datos que recibe del servidor por pantalla a través de la interfaz gráfica.

Seguidamente se detalla la estructura de cada una de las partes principales del sistema.

4.3.1 Back-end

El back-end es la parte servidor de la aplicación, encargada de recibir las solicitudes que los usuarios realizan a través del cliente y de enviar las respuestas correspondientes. Está compuesto por numerosos ficheros, pudiendo agruparse en 3 categorías: componentes, configuración y utilidades. La parte del servidor también incluye la base de datos en la que se almacena la información de forma persistente y cuyo diseño puede encontrarse detallado en el apartado ‘3.1 Modelo de datos y base de datos’.

El servidor cuenta con un fichero especial que se encuentra en el directorio raíz del código, de nombre ‘main.js’, que se corresponde con el fichero de ejecución principal y punto de entrada a la API. La estructura global del código del servidor queda reflejada en la Ilustración 12.

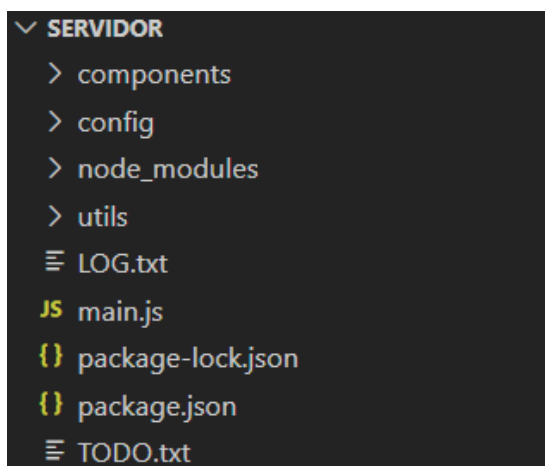


Ilustración 12. Estructura del código del servidor.

Componentes

Los componentes representan todos los recursos de los que dispone el sistema. Contienen el código necesario para permitir las funcionalidades definidas en base a los requisitos del sistema. En este caso, y por la forma en que está diseñado el sistema, existen 4 componentes: reserva, sala, sesión y usuario. El contenido de esta carpeta puede observarse en la Ilustración 13.

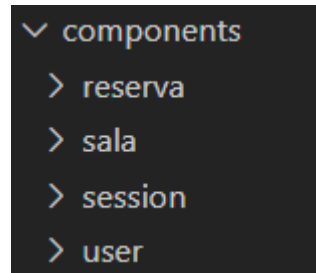


Ilustración 13. Carpeta componentes del servidor.

Cada subcarpeta contiene a su vez los ficheros de cada componente, como muestra la Ilustración 14.

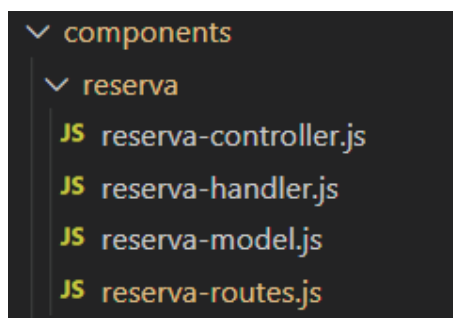


Ilustración 14. Contenido de la carpeta de un componente.

Seguidamente, se detalla cada uno de los ficheros.

- Routes: fichero encargado de definir las rutas diseñadas de la API. Es el encargado de dirigir las peticiones de entrada al lugar en el que son tratadas. Es en este fichero donde se establece la protección de la ruta, que permite o deniega el acceso en función de los permisos del usuario. La Ilustración 15 muestra un posible contenido de este fichero, en este caso del componente de reserva. En la imagen podemos ver que para una petición sobre el punto de acceso '/reservas/nueva' (línea 32 de la Ilustración 15), se realizan una serie de comprobaciones previas para posteriormente pasar la petición a la función correspondiente.

```

23 // [/reservas] GET ALL RESERVAS
24 router.get('/', jwt_utils.tokenExists, jwt_utils.validateToken, jwt_utils.isAdmin, ReservaController.reservas_get_all);
25 // [/reservas/:reserva_id] GET ONE RESERVA
26 router.get('/:reserva_id', jwt_utils.tokenExists, jwt_utils.validateToken, ReservaController.reservas_get_one);
27 // [/reservas/salas/:sala_id]
28 router.get('/:salas/:sala_id', jwt_utils.tokenExists, jwt_utils.validateToken, ReservaController.reservas_get_all_sala);
29 // [/reservas/:reserva_id] EDIT UNA RESERVA
30 router.put('/:reserva_id', jwt_utils.tokenExists, jwt_utils.validateToken, ReservaController.reservas_edit);
31 // [/reservas/nueva] CREATE RESERVA
32 router.post('/:nueva', jwt_utils.tokenExists, jwt_utils.validateToken, ReservaHandler.checkReserva, ReservaController.reservas_create);
33 // [/reservas/:reserva_id] ELIMINAR UNA RESERVA
34 router.delete('/:reserva_id', jwt_utils.tokenExists, jwt_utils.validateToken, ReservaController.reservas_delete);
    
```

Ilustración 15. Fichero routes del componente reserva.

- Handler: fichero usado para realizar comprobaciones y asegurar que la petición realizada a la API tiene la estructura y contenido necesarios para ser atendida. En la Ilustración 16 se muestra una de estas comprobaciones, realizada sobre el cuerpo de la petición de la reserva, en la que se comprueba que existan todos los campos necesarios.

```

1  module.exports.checkReserva = (req,res,next) => {
2      const body = req.body;
3      if(body.userId && body.salaId && body.dia && body.hora_inicio && body.duracion){
4          next();
5      }else{
6          res.status(400).send({
7              status: 'failed',
8              message: 'Campos incompletos'
9          })
10         return;
11     }
12 }

```

Ilustración 16. Función del handler del componente reserva.

- Model: fichero que define la estructura y los atributos de las entidades de la base de datos. El fichero será utilizado por el ORM Sequelize para generar las diferentes tablas de la base de datos de forma automática.
- Controller: en este fichero se encuentra toda la lógica referente al componente. Recibe la petición y, tras realizar diferentes acciones, devuelve una respuesta y datos en caso de ser necesarios. Es además el encargado de comunicarse con la base de datos, para recuperar o registrar información nueva. La Ilustración 17 contiene un ejemplo de una de las funciones incluidas en este tipo de fichero, en este caso del componente reserva.

```

274 exports.reservas_delete = (req,res) => {
275     return db.transaction(transaction => {
276         return Reserva.destroy({ where: { id: req.params.reserva_id}, transaction: transaction})
277         .then(rowsAffected => {
278             res.status(200).send({
279                 status: 'success',
280                 message: 'Reserva eliminada.'
281             })
282             return;
283         }).catch(err => console.log('Error: '+err));
284     }).catch(err => console.log('Error: '+err));
285 }

```

Ilustración 17. Función del controller de reserva.

Configuración

Los ficheros de configuración se encargan de configurar las distintas tecnologías utilizadas. En este caso, configurar la conexión a las diferentes bases de datos utilizadas (MySQL y Redis). Las relaciones entre entidades de la base de datos se han incluido como fichero de configuración, aunque pueden definirse sobre los propios modelos de las entidades. La Ilustración 18 muestra el contenido de la carpeta configuración de este proyecto.

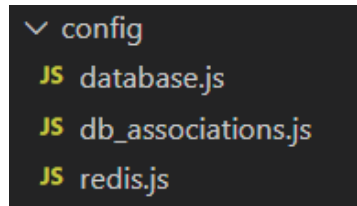


Ilustración 18. Carpeta de configuración del servidor.

Utilidades

Los ficheros de utilidad contienen funciones auxiliares que pueden ser necesarias en cualquier otra parte del código, como pueden ser una función para dar formato a una fecha o crear una dirección de correo a través de un nombre y un apellido. La Ilustración 19 muestra el contenido de esta carpeta.

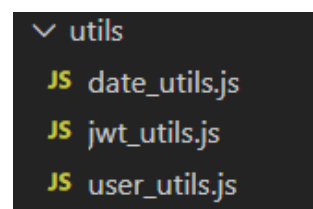


Ilustración 19. Carpeta de utilidades del servidor.

4.3.2 Front-end

El front-end es la parte cliente del sistema, la encargada de interactuar con el usuario, de enviar las solicitudes al servidor y de mostrar los datos que este le devuelve a través de la interfaz. Está compuesto por multitud de ficheros y, al igual que el servidor, queda dividido en varias carpetas. La estructura del código del cliente se muestra en la Ilustración 20.

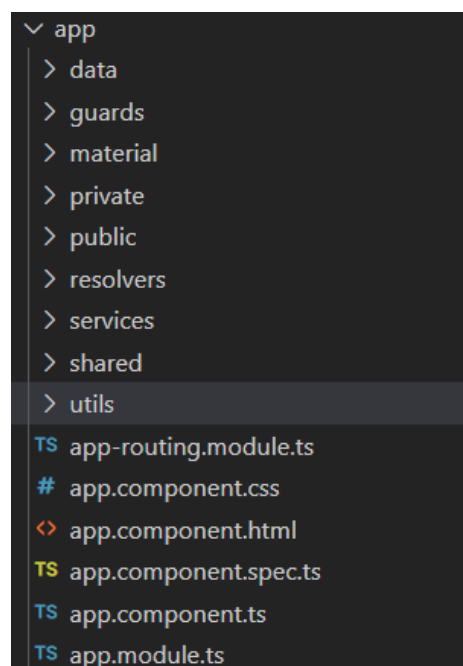


Ilustración 20. Estructura del código del cliente.

Antes de detallar el contenido de las diferentes carpetas, es necesario explicar los conceptos de componente y módulo. Un componente, al igual que en el servidor, es el encargado de dotar de una determinada funcionalidad al sistema. Y un módulo, es un conjunto de componentes que generalmente están relacionados.

Módulo

El sistema cuenta con un total de 4 módulos principales. El módulo principal de la aplicación, el módulo material y 2 módulos que diferencian las funcionalidades públicas de las privadas. En el módulo público se encuentran todos los componentes que pueden ser accedidos por cualquier usuario como son el *home* o el *login*. En el módulo privado se encuentran los componentes que únicamente usuarios registrados y autorizados pueden acceder, como lo es el componente de *reserva*. En la Ilustración 21 se muestra el contenido del módulo público.

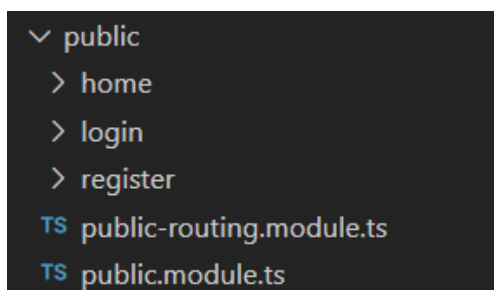


Ilustración 21. Contenido del módulo público.

Dentro de cada módulo, el fichero nombrado '*<nombre-del-modulo>.module.ts*' contiene la declaración de todos los componentes que lo forman. En el caso de la ilustración superior, como representa el módulo público, estarán declarados el componente *home*, *login* y *register*.

Al declarar los componentes en el módulo, siempre que se quiera utilizar cualquiera de ellos de nuevo, basta con incluir el módulo en el que ha sido declarado. Es el caso del módulo 'material', que contiene declarados todos los componentes de Angular Material, y que, tras declararse en el módulo de la aplicación (*app.module.ts*), sus componentes pueden usarse en cualquier parte del código.

Componente

El componente contiene el código de una determinada funcionalidad del sistema, como puede ser mostrar una lista con las diferentes reservas realizadas por un usuario. Cada componente está compuesto por 4 ficheros:

- *component.ts*: fichero Typescript que contiene el código que otorga la lógica al componente.
- *component.html*: contiene el código HTML que indica al navegador web la estructura y posicionamiento de los diferentes contenidos en la página web.

- component.css: contiene el código en lenguaje CSS con los estilos que van a ser aplicados sobre el fichero HTML.
- component.spec.ts: fichero utilizado para realizar pruebas sobre el componente.

El contenido de un componente queda reflejado en la Ilustración 22.

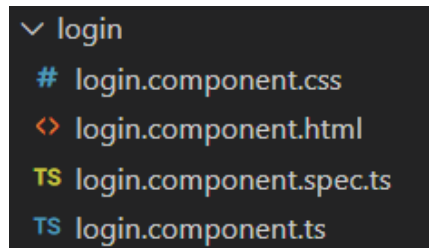


Ilustración 22. Ficheros de un componente.

Este elemento del front-end sigue el modelo de vista controlador (MVC). En él, la lógica queda separada de los datos y su presentación.

El modelo vista controlador está compuesto por 3 partes: el modelo, la vista y el controlador. La vista es la encargada de la representación de los datos y se corresponde con los ficheros HTML y CSS, que contienen la estructura y estilo de los datos a representar. El controlador es el encargado de la lógica y de la comunicación entre la vista y el modelo, pues estos no tienen interacción directa entre ellos. El controlador se corresponde con el fichero typescript. Por último, está el modelo, responsable de los datos y que se corresponde con el fichero de servicio, explicado a continuación.

Servicio

El servicio es la parte del código encargada de otorgar los datos al componente, y se corresponde con la parte del modelo en el modelo vista controlador. Es también el encargado de comunicarse con el servidor mediante peticiones HTTP para enviar o recibir datos en función de la solicitud realizada por el usuario. La Ilustración 23 muestra un ejemplo de un método del fichero de servicio de salas del sistema. En este caso, el método se corresponde con la creación de una nueva sala para reservar. En él, los datos de la sala se envían en el cuerpo de la petición para su registro en la base de datos.

```

35  ✓ createSala(sala: any): Observable<any> {
36      const url = this.salasUrl;
37  ✓  return this.http.post<any>(url, sala, this.httpOptions).pipe(
38      |      catchError(this.handleError<any>('createSala'))
39      )
40  }
```

Ilustración 23. Método del fichero servicio de salas.

Interceptor

El interceptor es un tipo de servicio especial que se encarga de capturar, tanto las peticiones antes de ser enviadas al servidor, como las respuestas recibidas

antes de llegar al componente. En el caso de las peticiones al servidor, se encarga de incluir el token de acceso a la API, explicado más adelante. En las respuestas, en caso de recibir algún código de error por parte del servidor, es el encargado de notificarlo.

Routing

El fichero routing contiene las instrucciones que indican qué componentes cargar en función de la ruta del cliente. Por ejemplo, para una ruta de '/login', el routing se encargará de indicar la carga del componente *login*. Hay un total de 3 ficheros de routing en el cliente: uno general, uno para la parte pública y otro para la parte privada. El contenido principal de un fichero routing son las rutas, que pueden protegerse estableciendo guardias, explicados a continuación. La Ilustración 24 muestra unas posibles rutas del fichero de routing.

```

7  const publicRoutes: Routes = [
8    { path: 'login', component: LoginComponent},
9    { path: 'register', component: RegisterComponent},
10   { path: 'home', component: HomeComponent}
11  ];

```

Ilustración 24. Rutas del fichero de routing.

Guardia

Un guardia o guard se encarga de la protección de las rutas. Cuando un usuario intenta acceder a una ruta, el guard comprueba que este tenga los permisos necesarios para acceder. En caso contrario, bloquea el acceso e informa al usuario. En la Ilustración 25 se muestran varias rutas protegidas con guards. Las dos primeras tienen únicamente protección de autenticación (haber iniciado sesión en el sistema y que esta sea válida) mientras que la tercera comprueba también que el usuario sea un administrador.

```

18  { path: 'dashboard', component: DashboardComponent, canActivate: [AuthGuard]},
19  { path: 'salas', component: SalasListComponent , canActivate: [AuthGuard]},
20  { path: 'salas/nueva', component: SalasCreateComponent , canActivate: [AuthGuard, AdminAuthGuard]},

```

Ilustración 25. Rutas con guards.

Resolver

El resolver es el encargado de asegurar que los datos han sido recibidos del servidor antes de la carga de un determinado componente. Es esencial en aquellos componentes en los que la carga depende de los datos para evitar errores. Al igual que los guards, se incluyen en el fichero de routing en las rutas necesarias.

4.4 Seguridad

En este subapartado se hace referencia a ciertos aspectos relacionados con la seguridad que se han implementado en el sistema.

4.4.1 JWT

JWT [21] (JSON Web Token) es un estándar abierto basado en JSON [22] propuesto por la IETF [23] (Internet Engineering Task Force) para la creación de tokens de acceso. Un token es un elemento emitido por una entidad privada que da a su propietario ciertos privilegios. En el caso de este trabajo, privilegios de acceso para hacer uso de diferentes funcionalidades del sistema de reserva, lo que se conoce como protección de rutas.

Este token es usado para la transmisión de información de forma segura entre dos partes, siendo la información enviada como un objeto JSON. Para asegurar su veracidad e integridad, se hace uso de una comprobación mediante firma digital, pudiendo firmarse mediante un secreto y el algoritmo HMAC, o utilizando un par clave pública-privada y RSA, entre otros algoritmos de cifrado.

El token JWT tiene una estructura compuesta por 3 partes claramente diferenciadas: Cabecera o Header, Carga o Payload y Firma o Signature. La estructura puede observarse en la Ilustración 26.

xxxxx.yyyyy.zzzzz

Ilustración 26. Estructura de un token JWT. [21]

Cabecera

La cabecera a su vez se divide en dos partes: el tipo de token y el algoritmo que se ha usado para la firma. Esta información está cifrada en Base64Url. En la Ilustración 27 puede observarse una representación de esta parte del token.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Ilustración 27. Cabecera de JWT. [21]

Carga

La carga es la parte del token que contiene los datos que quieren intercambiarse de forma segura entre las partes. Puede contener campos como el tiempo de expiración del token (exp) o el emisor del token (iss). Es aquí donde el usuario que emite el token introduce los datos que desea intercambiar. Al igual que la cabecera, el contenido es cifrado en Base64Url. En la Ilustración 28 puede observarse un posible contenido para esta parte.


```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

Ilustración 28. Carga de JWT. [21]

Firma

La parte de la firma se utiliza para comprobar que el token no ha sido alterado durante el envío. Para firmar se usa un algoritmo con un secreto. El contenido a cifrar se corresponde con la suma del contenido de la cabecera y de la carga, ambos cifrados en Base64Url, como puede verse en la Ilustración 29.

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret)
```

Ilustración 29. Firma de JWT. [21]

Una vez codificadas las tres partes, se construye el token que tiene la forma que representa la Ilustración 30.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaXNjb2NpYWwiOiJpvaG4gRG9lIiwiaXNjb2NpYWwiOiJpvaG4
4pcPyMD09olPSyXnrXCjTwXyr4BsezdI1AVTmud2fU4
```

Ilustración 30. Token JWT. [21]

La parte de rojo se corresponde con el cifrado de la cabecera, la parte en morado con el cifrado de la carga (los datos) y la parte de azul con el cifrado resultante tras aplicar el algoritmo de la firma con el secreto. Las diferentes partes quedan separadas por un punto para facilitar su división.

Una vez formado, el token deberá incluirse en la cabecera 'Authorization' de la petición HTTP con el formato de la Ilustración 31.

```
Authorization: Bearer <token>
```

Ilustración 31. Formato de cabecera 'Authorization' para token JWT. [21]

De este modo, el sistema reconocerá que se trata de un token JWT y comenzará a realizar las comprobaciones necesarias para el acceso del usuario.

El usuario obtendrá este token tras iniciar sesión en el sistema. Este proceso de obtención del token queda reflejado en la Ilustración 32.

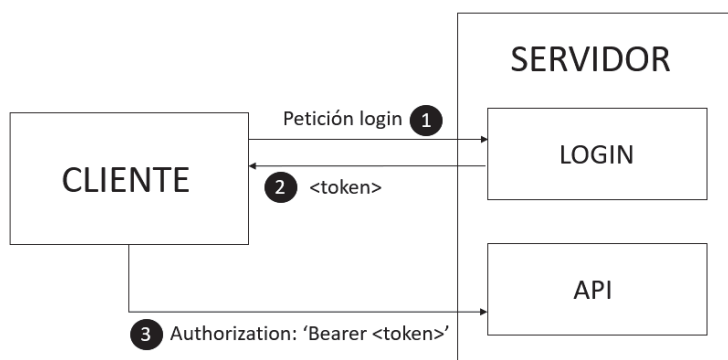


Ilustración 32. Proceso de obtención del token JWT.

Una vez que el usuario recibe el token, el propio sistema se encarga de incluirlo en la cabecera 'Authorization' mediante el Interceptor (ver apartado '4.3.2 Front-end') con el formato antes mencionado para las realizar diferentes peticiones.

En el momento en el que el usuario inicia sesión, el token quedará registrado en la base de datos en memoria Redis, en la que tendrá un tiempo de vida determinado tras el que será eliminado. Este tiempo de vida se corresponde con el tiempo de vida de la sesión del usuario. En caso de realizar una consulta con un token expirado, el usuario será informado de ello y redirigido a la pantalla de login para crear una nueva sesión.

4.4.2 Base de datos: MySQL

Con el objetivo de asegurar la integridad de la información almacenada en la base de datos, se han establecido algunos mecanismos de seguridad. Estos mecanismos son las transacciones, las restricciones o constraints y la selección exclusiva de datos.

Transacciones

Una transacción, en base de datos, hace referencia a una o varias operaciones que se realizan sobre la base de datos de forma secuencial y sin ser visibles para el resto de las conexiones. Serán visibles únicamente cuando se realice un 'commit' o confirmación de operación, momento en el que se harán efectivos los cambios realizados sobre la base de datos.

La peculiaridad que tienen las transacciones es que, en caso de producirse un error en cualquiera de las operaciones de la transacción, esta queda cancelada, haciendo que los cambios que se iban a realizar no se efectúen, es decir, no se llega a realizar el 'commit' de confirmación.

Este mecanismo ha sido utilizado en la mayoría de las operaciones realizadas sobre la base de datos del sistema.

Restricciones

Las restricciones o constraints son un mecanismo de seguridad que establece una serie de reglas para las diferentes tablas de la base de datos. Se aplican sobre una o varias columnas y son una forma de mantener la integridad y confiabilidad de los datos almacenados.

En el caso de este proyecto, para asegurar que no se producía duplicidad de reservas, se ha creado una restricción de nombre 'reserva_unica' que asegura que no haya dos reservas en las que coincidan los campos de 'dia', 'hora_inicio' y 'salaId' (ver descripción de reserva en el apartado '3.1 Modelo de datos y base de datos'). En caso de intentar crear una reserva con datos ya existentes en la base de datos, el sistema devuelve un error.

Se ha establecido también una restricción para los campos de 'username' y 'email' (ver descripción de usuario en el apartado '3.1 Modelo de datos y base de datos') de forma que no se permitan nombres de usuario o email duplicados.

Selección exclusiva

Con el objetivo de evitar eliminar un registro de datos que está siendo actualizado, se ha implementado una funcionalidad disponible en MySQL: 'select for update'. Con esta funcionalidad, siempre que se seleccione una determinada entrada de una tabla, esta queda bloqueada para el resto de las conexiones hasta que la conexión que la ha bloqueado finaliza su transacción, impidiendo por tanto cualquier tipo de modificación o eliminación del registro por parte de aquellas conexiones que no la han bloqueado.

Cifrado de contraseña

Con el objetivo de incrementar la seguridad del sistema y de evitar daños en caso de un filtrado de datos, las contraseñas de los diferentes usuarios registrados en el sistema se almacenan cifradas, haciendo uso de una función hash. Cuando el usuario inicia sesión, para comprobar que la contraseña introducida coincide con la almacenada, se aplica la misma función hash sobre la contraseña introducida y se compara. En caso de ser iguales, se permite el inicio de sesión al usuario. Este mismo mecanismo se utiliza para el cambio de contraseña, en el que se comprueba que la contraseña antigua coincida con la almacenada.

4.4.3 Funcionalidad

Se han implementado también ciertos mecanismos de seguridad en cuanto a funcionalidad del sistema.

Uno de ellos es la doble comprobación de la contraseña. Cuando un usuario se registra o quiere cambiar la contraseña, el sistema comprueba que la contraseña introducida se corresponde con el campo de confirmación de la contraseña. En caso de que estos campos no coincidan, el sistema impide el envío del formulario.

La protección de las rutas es otro de ellos. Cuando un usuario intenta acceder a una ruta protegida, se realizan una serie de comprobaciones. En caso de que no cumpla alguno de los requisitos para acceder, se deniega su acceso y se le informa. El sistema cuenta con un total de 3 tipos de comprobaciones: que el usuario tenga sesión activa, que el usuario sea administrador y que el usuario sea él mismo. Esta última es utilizada únicamente para permitir el acceso del usuario a sus propios recursos y no al del resto de usuarios del sistema.

5 Pruebas

Para cada etapa del desarrollo, se realizaron numerosas pruebas para asegurar el correcto funcionamiento del sistema, de modo que se pudiese asegurar su correcto funcionamiento.

Se realizaron un total de 3 procesos de pruebas: uno para probar el backend, otro para el frontend, y un último proceso para probar el funcionamiento global del sistema.

Backend

Para realizar las pruebas de esta parte del desarrollo, se utilizó la herramienta Postman, que permite realizar peticiones HTTP sin la necesidad de contar con un cliente.

A continuación, se muestra un ejemplo de la estructura de una de las pruebas realizadas. En este caso, se trata del inicio de sesión de un usuario. Como podemos ver en la Ilustración 33, en el cuerpo se encuentran los campos necesarios para realizar esta petición, el usuario y la contraseña. La petición debe realizarse sobre el punto de acceso de la API que contiene el servicio, en este caso sobre '/login'.

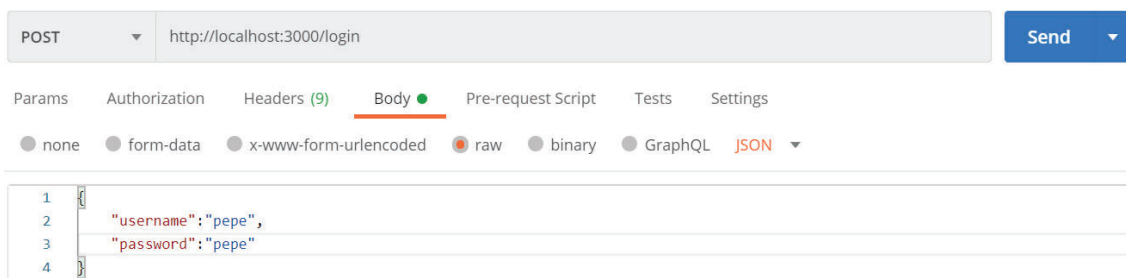


Ilustración 33. Cuerpo prueba petición inicio de sesión.

Tras enviar la petición al servidor, este envía una respuesta que puede verse en la Ilustración 34.

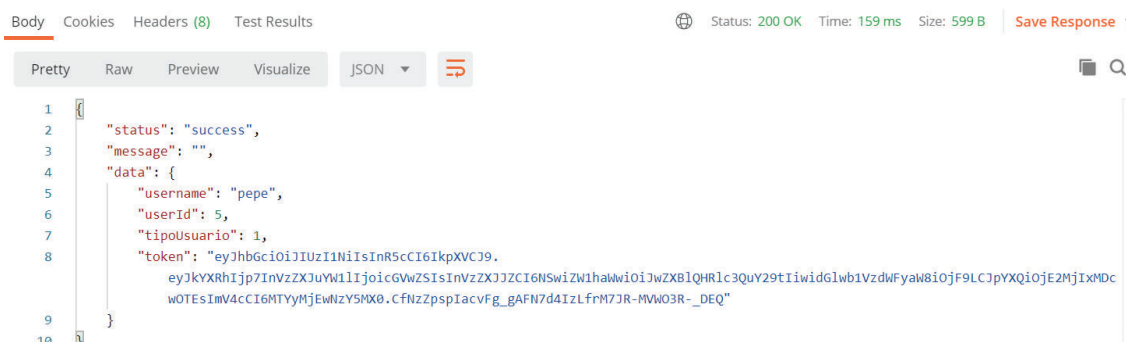


Ilustración 34. Respuesta prueba inicio de sesión.

La respuesta recibida desde el servidor indica que la petición se ha llevado a cabo correctamente, ya que tiene un status de 200 OK. En el cuerpo de la respuesta se observa, además, el resto de información que el servidor envía

como el tipo de usuario del usuario que ha iniciado sesión o el token de acceso que necesitará para acceder a los diferentes servicios del sistema web.

La prueba realizada, al tratarse del inicio de sesión, no requería de ningún tipo de token de acceso. En caso de realizar una prueba sobre un recurso protegido, deberá incluirse el token de tipo 'Bearer' en la cabecera 'Authorization', como muestra la Ilustración 35.

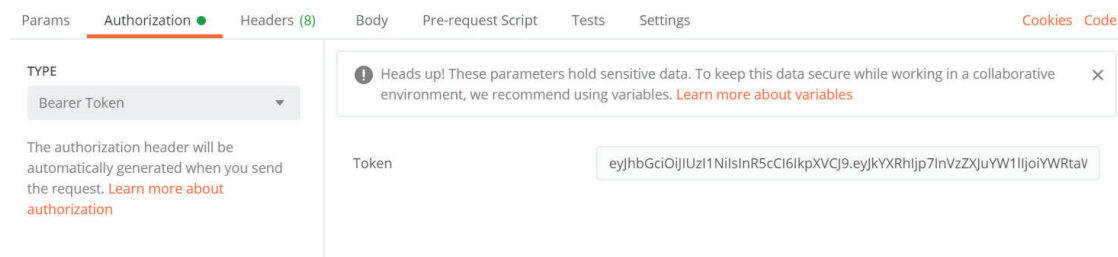
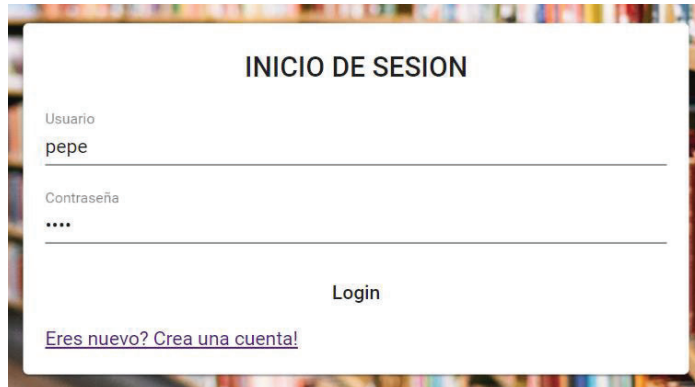


Ilustración 35. Autorización con token para las peticiones de las pruebas.

Frontend

Para probar la parte cliente, se utilizó la consola del navegador web Google Chrome. Antes de enviar la petición al servidor, se imprimía por la consola del navegador tanto el contenido del cuerpo enviado al servidor como la URI a la que se enviaba.

Para, por ejemplo, probar el inicio de sesión del sistema, se introducían los datos en el formulario de inicio de sesión, como muestra la Ilustración 36.



The screenshot shows a login form titled 'INICIO DE SESION'. It has two input fields: 'Usuario' with the value 'pepe' and 'Contraseña' with masked characters '....'. Below the fields is a 'Login' button. At the bottom, there is a link that says 'Eres nuevo? Crea una cuenta!'.

Ilustración 36. Formulario de prueba de inicio de sesión.

Tras enviar la petición, en la consola del navegador aparece tanto el cuerpo como la URI sobre la que se envía.

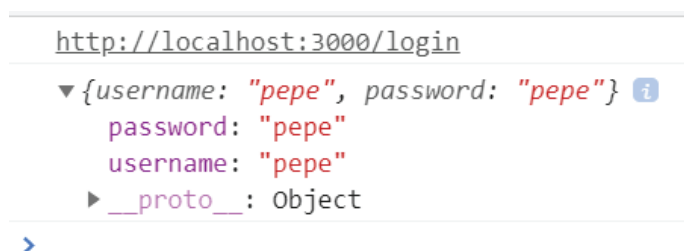


Ilustración 37. Cuerpo y URI enviados al servidor.



También se realizaron pruebas de seguridad y de interfaz. Las primeras para comprobar que los usuarios no pueden acceder a funcionalidades para las que no tienen permisos, y las segundas, para comprobar que ciertas partes de la interfaz aparecen únicamente cuando el usuario es de tipo administrador, como los botones para editar o eliminar salas.

Global

Por último, para comprobar el funcionamiento general del sistema, se probaron las diferentes funcionalidades que este ofrece, tanto para un usuario básico como un usuario administrador. Se realizaron además test con familiares y amigos simulando potenciales clientes.

6 Resultados y conclusiones

Este proyecto permitirá a numerosos negocios iniciar su proceso de transformación digital gracias a la aplicación web. En caso de haber iniciado ya la transformación digital, pero de no disponer de un sistema de reserva web, esta puede ser una forma de satisfacer esa necesidad. El sistema no solo es viable para un tipo de reserva y negocio determinados, ya que, aunque en este caso la aplicación sea de reserva de salas de una biblioteca, gracias a su diseño basado en módulos permite su adaptación a otro tipo de sistemas de reservas como pueden ser restaurantes, habitaciones de hoteles o vehículos, realizando escasas modificaciones.

El sistema desarrollado cumple con la totalidad de requisitos propuestos en la etapa de planificación, además del objetivo principal de diseño. Este objetivo era que la aplicación tuviese disponible toda la información en pantalla a través de una interfaz sencilla y amigable, que ha sido posible probar gracias a la colaboración de amigos y familiares. El aspecto final de la pantalla de login del sistema de reserva puede observarse en la Ilustración 38, pudiendo encontrar el resto en el ‘Anexo 2 – Interfaz web’.

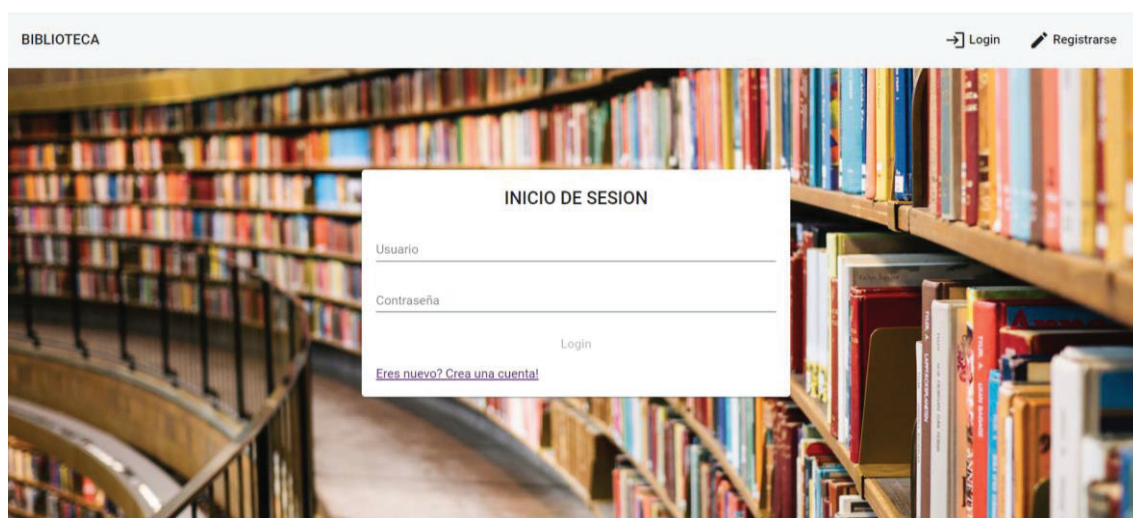


Ilustración 38. Pantalla de login.

Respecto al proyecto, se han podido alcanzar todos los objetivos propuestos en un principio, a pesar de las dificultades encontradas durante el desarrollo. No ha sido posible en el caso de la planificación, ya que por diversas circunstancias se ha retrasado, no pudiendo cumplirse la planificación establecida en un primer momento (ver apartado ‘1.2 Planificación’).

En general, estoy satisfecho con el resultado obtenido. Sin embargo, siempre existen aspectos que se pueden mejorar. En este caso, los próximos pasos a seguir en relación con el desarrollo del sistema serían el despliegue tanto del cliente como del servidor, para permitir el uso y acceso a la aplicación desde cualquier ordenador que disponga de una conexión a internet y un navegador web. Otra de las tareas a realizar en un futuro sería la de adaptar el código de la interfaz del cliente para hacerlo dinámico y que este pueda adaptarse a



dispositivos con pantallas de cualquier tamaño, ya que actualmente el funcionamiento y estilo está pensado únicamente para ordenadores.

Por último, quiero indicar que la elaboración de este trabajo me ha permitido afianzar muchos de los conocimientos adquiridos a lo largo de la carrera, al haberlos aplicado durante el proceso de desarrollo (diseño de APIs, BBDD, etc.), pero, sobre todo, me ha permitido adquirir y ampliar conocimientos en una nueva rama de la informática como es el desarrollo de aplicaciones y servicios web. También he podido profundizar en las principales tecnologías utilizadas para el desarrollo de dichas aplicaciones, que espero me sirva en un futuro cercano para disponer de un amplio abanico de posibilidades en el mundo laboral.

7 Análisis de impacto

En septiembre del año 2015, 193 países, entre ellos España, se comprometieron a seguir los 17 Objetivos de Desarrollo Sostenible [24] propuestos por las Naciones Unidas y teniendo como objetivo su cumplimiento para el año 2030.

Estos objetivos persiguen la igualdad entre las personas, proteger el planeta y asegurar la prosperidad de la sociedad.

Algunos de estos objetivos son el fin de la pobreza, erradicar el hambre en el mundo, la salud y el bienestar o la educación de calidad. De todos ellos, este trabajo persigue el objetivo número 9: Industria, Innovación e Infraestructura [25] que busca construir infraestructuras con capacidad de adaptación, promover la industrialización inclusiva y sostenible y fomentar la innovación.

Principalmente, este trabajo contribuye en los aspectos de sostenibilidad e innovación.

Con este proyecto, se pretende facilitar el proceso de transformación digital a aquellas empresas que todavía no han iniciado el proceso a través de la creación de un sistema que les permita digitalizar y modernizar el proceso de reserva con el que cuentan. Concretamente en este caso, un sistema para la reserva de las salas de una biblioteca. De este modo se pretende incrementar la eficiencia del establecimiento evitando los ya atrasados métodos habituales de reserva como lo son las llamadas de teléfono al establecimiento. Aunque es igualmente aplicable sobre otro tipo de situaciones que requieran realizar una reserva como puede ser un restaurante.



8 Bibliografía

- [1] Javascript - <https://developer.mozilla.org/es/docs/Web/JavaScript>
- [2] Typescript - <https://www.typescriptlang.org/>
- [3] Html - <https://developer.mozilla.org/es/docs/Web/HTML>
- [4] CSS - <https://developer.mozilla.org/es/docs/Web/CSS>
- [5] Node.js - <https://nodejs.org/es/>
- [6] Express – Infraestructura de aplicaciones web Node.js - <https://expressjs.com/es/>
- [7] Sequelize - <https://sequelize.org/master/index.html>
- [8] MySQL - <https://www.mysql.com/>
- [9] ¿Qué es una base de datos relacional?, Oracle - <https://www.oracle.com/es/database/what-is-a-relational-database/>
- [10] MySQL Workbench - <https://www.mysql.com/products/workbench/>
- [11] NPM - <https://www.npmjs.com/>
- [12] Redis - <https://redis.io/>
- [13] ¿Qué es Redis? - <https://aws.amazon.com/es/elasticache/what-is-redis/>
- [14] Github - <https://kinsta.com/es/base-de-conocimiento/que-es-github/>
- [15] Git - <https://git-scm.com/>
- [16] Postman - <https://www.postman.com/>
- [17] VSCode - <https://code.visualstudio.com/>
- [18] Introduction to JSON Web Token - <https://jwt.io/introduction>
- [19] Introducción a JSON - <https://www.json.org/json-es.html>
- [20] IETF - <https://www.ietf.org/>
- [21] base64url - <https://www.base64url.com/>
- [22] Angular - <https://angular.io/>
- [23] Angular Material - <https://material.angular.io/>
- [24] Modelo Vista Controlador (MVC) - <https://desarrolloweb.com/articulos/que-es-mvc.html>
- [25] Desarrollo Sostenible - <https://www.agenda2030.gob.es/objetivos/home.htm>
- [26] Objetivo 9: Industria, Innovación e Infraestructura - <https://www.agenda2030.gob.es/objetivos/objetivo9.htm>

9 Anexos

9.1 Anexo 1 – Diseños de interfaz



Title

BIBLIOTECA

INICIO DE SESION

Usuario

Contraseña

Login

[Eres Nuevo? Crea una cuenta!](#)

Ilustración 39. Diseño de pantalla de inicio de sesión.



Title

BIBLIOTECA

REGISTRO

Usuario

Contraseña

Confirmar contraseña

Email

Nombre

Registro

[Ya tienes cuenta? Inicia sesión!](#)

Ilustración 40. Diseño de pantalla de registro de usuario en el sistema.



Ilustración 41. Diseño de pantalla dashboard.

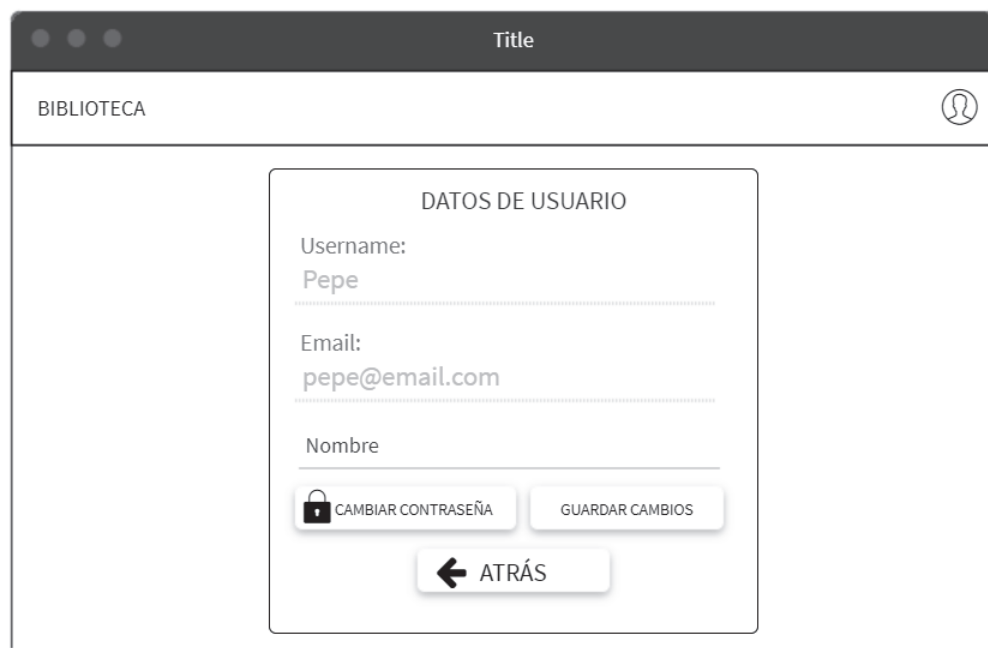


Ilustración 42. Diseño de pantalla de datos de usuario.



Title

BIBLIOTECA

CAMBIAR CONTRASEÑA

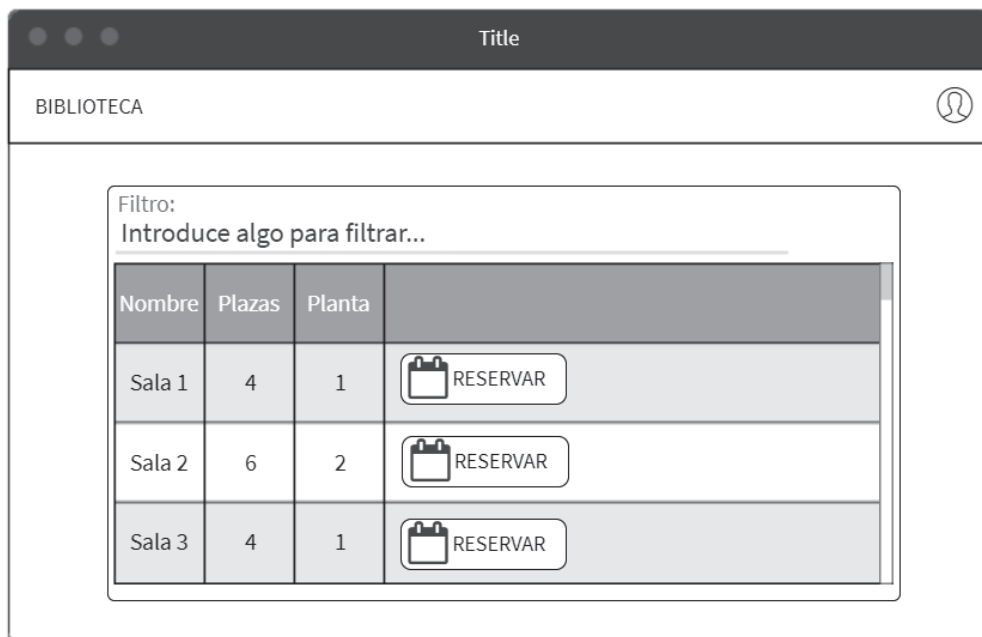
Contraseña actual:

Contraseña nueva:

Confirmar contraseña nueva:

GUARDAR ← ATRÁS

Ilustración 43. Diseño de pantalla de cambio de contraseña.



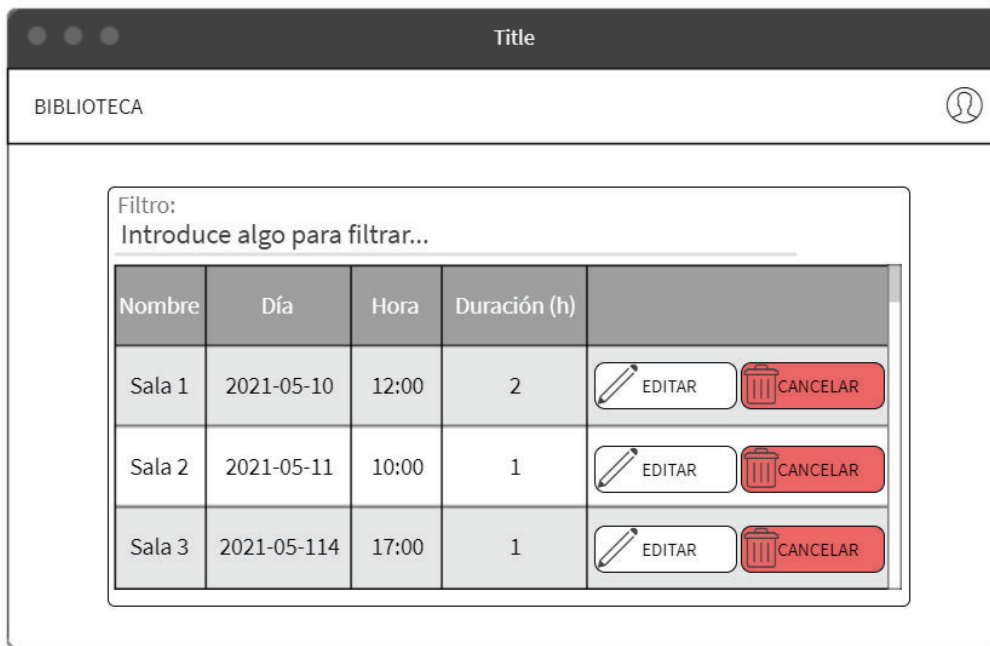
Title

BIBLIOTECA

Filtro:
Introduce algo para filtrar...

Nombre	Plazas	Planta	
Sala 1	4	1	 RESERVAR
Sala 2	6	2	 RESERVAR
Sala 3	4	1	 RESERVAR

Ilustración 44. Diseño de pantalla de listado de salas



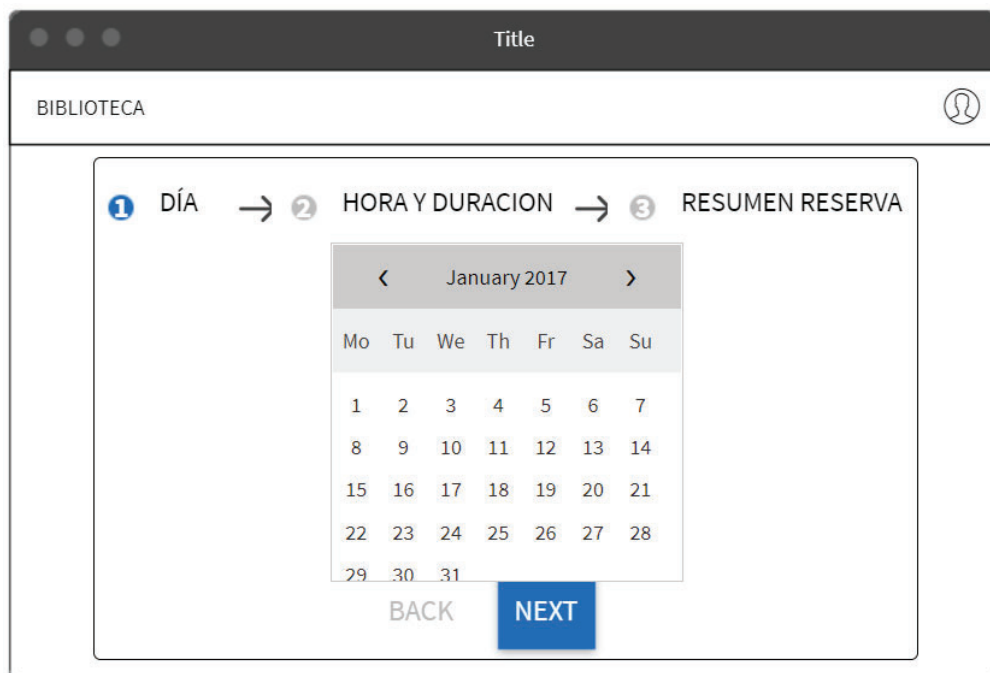
Title

BIBLIOTECA

Filtro:
Introduce algo para filtrar...

Nombre	Día	Hora	Duración (h)	
Sala 1	2021-05-10	12:00	2	EDITAR CANCELAR
Sala 2	2021-05-11	10:00	1	EDITAR CANCELAR
Sala 3	2021-05-11	17:00	1	EDITAR CANCELAR

Ilustración 45. Diseño de pantalla de listado de reservas



Title

BIBLIOTECA

1 DÍA → 2 HORA Y DURACION → 3 RESUMEN RESERVA

< January 2017 >

Mo	Tu	We	Th	Fr	Sa	Su
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

BACK NEXT

Ilustración 46. Diseño de pantalla de reserva



Ilustración 47. Diseño de pantalla de error 404 - elemento no encontrado

9.2 Anexo 2 – Interfaz web



Ilustración 48. Pantalla home.

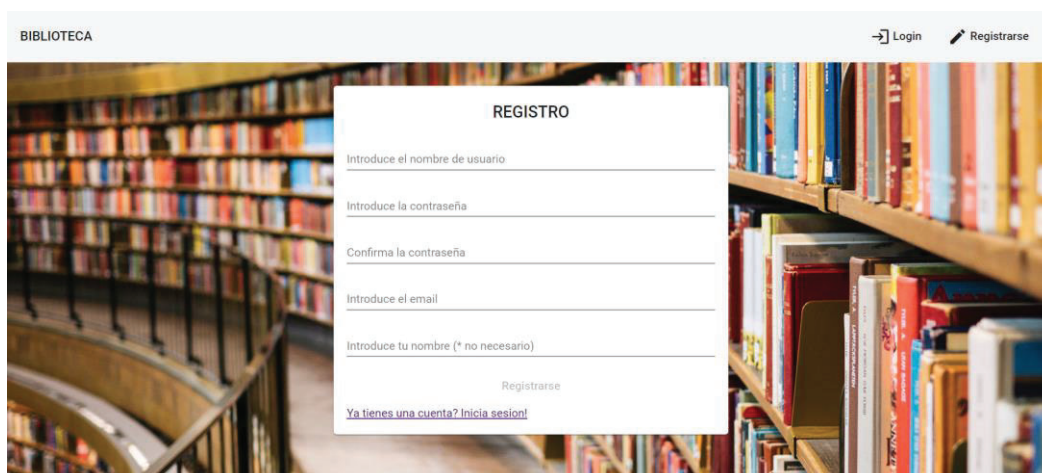


Ilustración 49. Pantalla de registro.

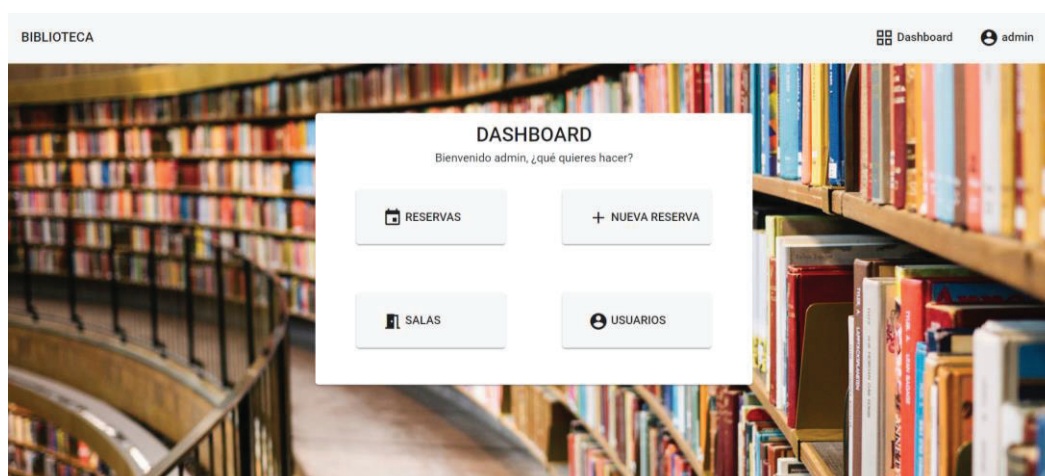


Ilustración 50. Pantalla dashboard.

BIBLIOTECA Dashboard admin

MIS RESERVAS

Filtro

Reserva	Día	Hora de inicio	Duración (h)	Sala	Id sala	Usuario		
Test cambio reserva	2021-04-25	17:00:00	2	test	1	pepe	EDITAR	CANCELAR
Test mala reserva	2021-04-26	17:00:00	1	test	1	test	EDITAR	CANCELAR
Test reserva 2	2021-04-27	17:00:00	1	test	1	test	EDITAR	CANCELAR
Test reserva 3	2021-04-28	17:00:00	1	test	1	test	EDITAR	CANCELAR

Items per page: 4 1 - 4 of 19 < >

[← Atrás](#)

Ilustración 51. Pantalla de reservas con vista de administrador.

BIBLIOTECA Dashboard admin

MIS RESERVAS

Filtro

Username	Tipo de usuario	Email	Nombre		
admin	Administrador	admin@test.com	Admin	EDITAR	ELIMINAR
test	Basico	test@test.com	test 4	EDITAR	ELIMINAR
pepe	Basico	pepe@test.com	pepe 10	EDITAR	ELIMINAR
luis	Basico	luis@email.com	luis 9	EDITAR	ELIMINAR
maria	Basico	maria@maria.com	Maria	EDITAR	ELIMINAR

Items per page: 5 1 - 5 of 9 < >

[← Atrás](#)

Ilustración 52. Pantalla de usuarios vista de administrador.

BIBLIOTECA Dashboard admin

SALAS

Filtro + AÑADIR SALA

Sala	Plazas	Planta				
test	4	1	RESERVAR	EDIT	DELETE	DESHABILITAR
test 2	6	2	RESERVAR	EDIT	DELETE	HABILITAR
test3	4	2	RESERVAR	EDIT	DELETE	DESHABILITAR
test4	6	1	RESERVAR	EDIT	DELETE	DESHABILITAR

Items per page: 4 1 - 4 of 15 < >

[← Atrás](#)

Ilustración 53. Pantalla de salas vista de administrador.

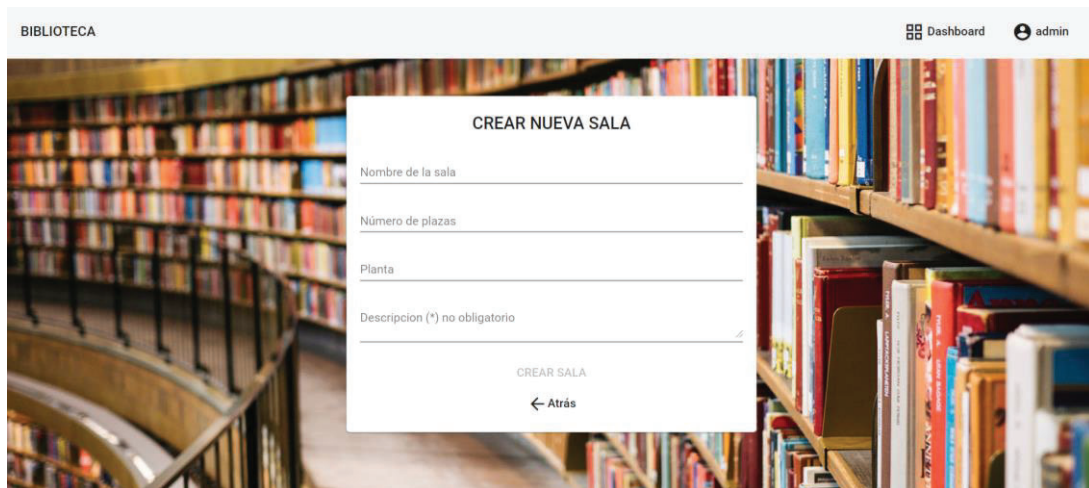


Ilustración 54. Pantalla de creación de sala.

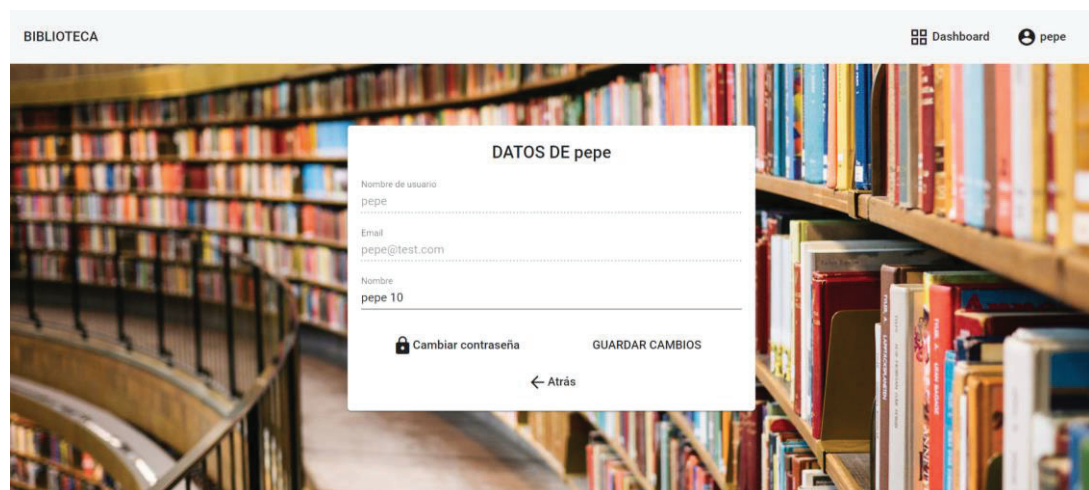


Ilustración 55. Pantalla de datos de usuario.



Ilustración 56. Pantalla de cambio de contraseña.

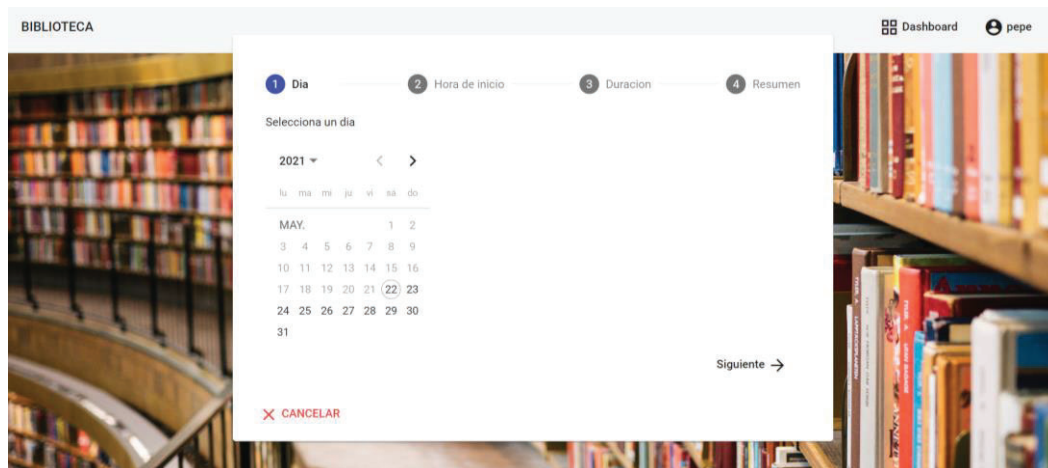


Ilustración 57. Pantalla de reserva.

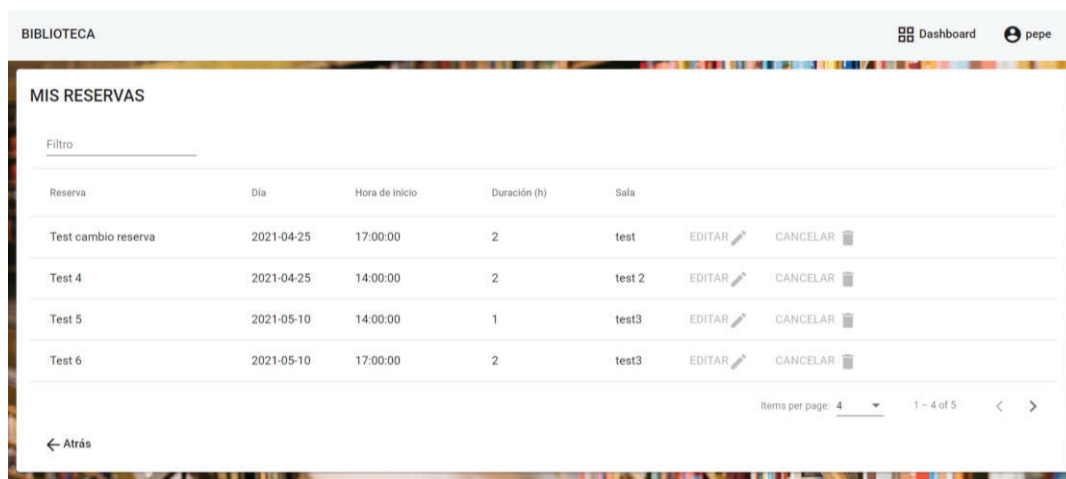


Ilustración 58. Pantalla de reservas de usuario.

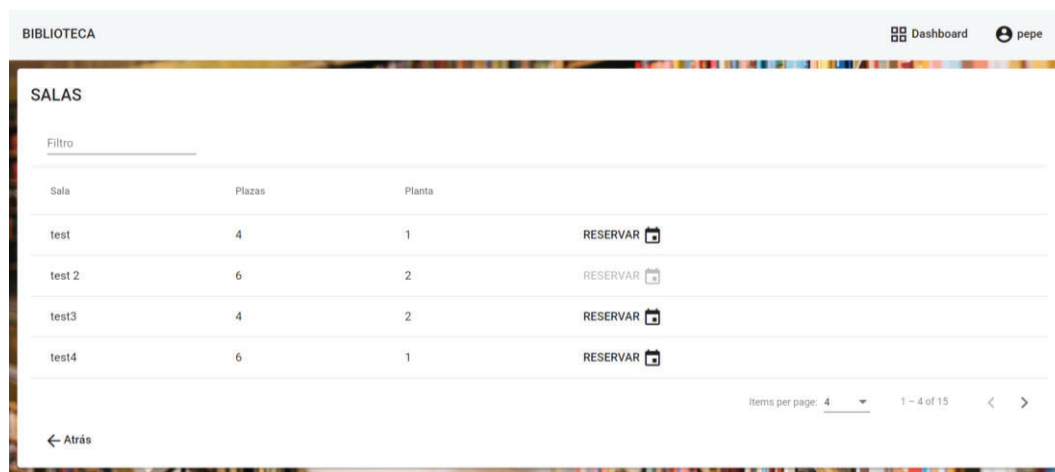


Ilustración 59. Pantalla de salas de usuario.

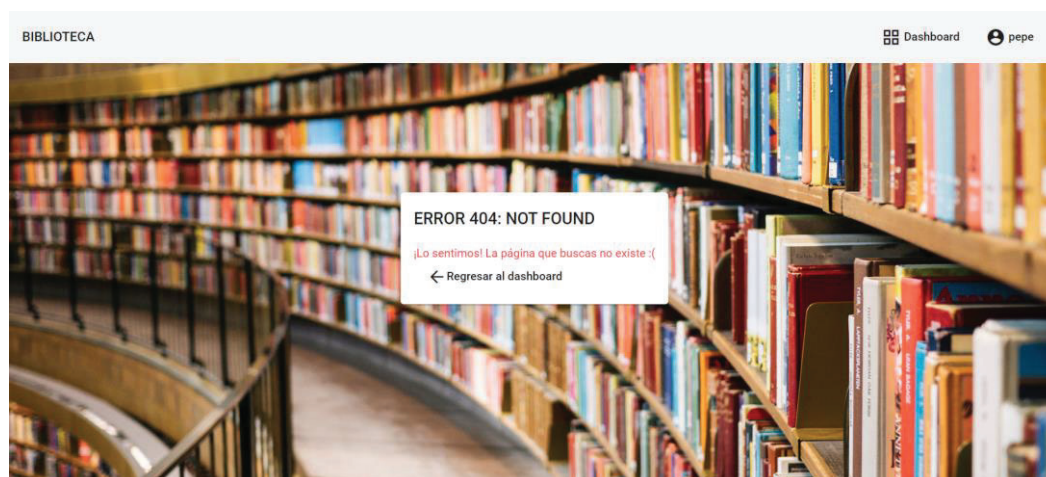


Ilustración 60. Pantalla de error 404 Not Found.

Este documento esta firmado por

	Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=Facultad de Informatica - UPM, C=ES
	Fecha/Hora	Wed Jun 02 16:56:37 CEST 2021
	Emisor del Certificado	EMAILADDRESS=camanager@fi.upm.es, CN=CA Facultad de Informatica, O=Facultad de Informatica - UPM, C=ES
	Numero de Serie	630
	Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)